Superagent Jroc AI — Production-Grade Starter Repo

Below is a complete multi-file **starter repository** for **Superagent Jroc AI**: a multi-page AI SaaS platform with auth, billing, agent builder, human verification marketplace, admin dashboard, and 50 sellable AI tools.

Stack: **Next.js 14 App Router**, **TypeScript**, **Tailwind CSS**, **Supabase**, **Stripe**, **OpenAI**

---

1) Repo Tree

```
superagent-jroc-ai/
├─ app/
│  ├─ (marketing)/
│  │  ├─ page.tsx
│  │  ├─ pricing/page.tsx
│  │  ├─ marketplace/page.tsx
│  │  ├─ automation-agents/page.tsx
│  │  ├─ human-verification/page.tsx
│  │  └─ layout.tsx
│  ├─ (auth)/
│  │  ├─ login/page.tsx
│  │  └─ signup/page.tsx
│  ├─ dashboard/
│  │  ├─ page.tsx
│  │  ├─ tools/page.tsx
│  │  ├─ agents/page.tsx
│  │  ├─ automation/page.tsx
│  │  ├─ verification/page.tsx
│  │  ├─ billing/page.tsx
│  │  ├─ settings/page.tsx
│  │  └─ api-keys/page.tsx
│  ├─ admin/
│  │  ├─ page.tsx
│  │  ├─ users/page.tsx
│  │  ├─ payments/page.tsx
│  │  ├─ workers/page.tsx
│  │  └─ analytics/page.tsx
│  ├─ api/
```

```
│   │   ├─ tools/run/route.ts
│   │   ├─ agents/create/route.ts
│   │   ├─ agents/run/route.ts
│   │   ├─ verification/jobs/route.ts
│   │   ├─ verification/worker-apply/route.ts
│   │   ├─ billing/checkout/route.ts
│   │   ├─ billing/portal/route.ts
│   │   ├─ stripe/webhook/route.ts
│   │   ├─ auth/session/route.ts
│   │   └─ health/route.ts
│   ├─ globals.css
│   └─ layout.tsx
├─ components/
│   ├─ marketing/
│   │   ├─ Hero.tsx
│   │   ├─ FeatureGrid.tsx
│   │   ├─ PricingCards.tsx
│   │   └─ ToolMarketplaceGrid.tsx
│   ├─ dashboard/
│   │   ├─ DashboardShell.tsx
│   │   ├─ Sidebar.tsx
│   │   ├─ Topbar.tsx
│   │   ├─ StatCard.tsx
│   │   ├─ ToolRunner.tsx
│   │   ├─ AgentBuilder.tsx
│   │   ├─ AutomationCanvas.tsx
│   │   ├─ VerificationJobForm.tsx
│   │   ├─ VerificationMarketplace.tsx
│   │   └─ BillingPanel.tsx
│   ├─ shared/
│   │   ├─ Logo.tsx
│   │   ├─ Button.tsx
│   │   ├─ Card.tsx
│   │   ├─ Input.tsx
│   │   └─ SectionHeading.tsx
│   └─ providers/
│       └─ AppProviders.tsx
├─ lib/
│   ├─ env.ts
│   ├─ supabase-server.ts
│   ├─ supabase-client.ts
│   ├─ auth.ts
│   ├─ stripe.ts
│   ├─ openai.ts
```

```
│   ├─ usage.ts
│   ├─ pricing.ts
│   ├─ permissions.ts
│   ├─ utils.ts
│   └─ validators.ts
├─ data/
│   ├─ ai-tools.ts
│   ├─ agent-templates.ts
│   └─ plans.ts
├─ db/
│   ├─ schema.sql
│   └─ seed.sql
├─ public/
│   └─ favicon.ico
├─ types/
│   ├─ tool.ts
│   ├─ agent.ts
│   ├─ billing.ts
│   └─ verification.ts
├─ middleware.ts
├─ package.json
├─ tsconfig.json
├─ tailwind.config.ts
├─ postcss.config.js
├─ next.config.mjs
├─ .env.example
└─ README.md
```

---

2) package.json

```
{
  "name": "superagent-jroc-ai",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint"
  },
  "dependencies": {
    "@supabase/ssr": "latest",
```

```
    "@supabase/supabase-js": "latest",
    "lucide-react": "latest",
    "next": "14.2.30",
    "openai": "latest",
    "react": "18.3.1",
    "react-dom": "18.3.1",
    "stripe": "latest",
    "zod": "latest"
  },
  "devDependencies": {
    "@types/node": "latest",
    "@types/react": "latest",
    "@types/react-dom": "latest",
    "autoprefixer": "latest",
    "postcss": "latest",
    "tailwindcss": "latest",
    "typescript": "latest"
  }
}
```

---

3) .env.example

```
NEXT_PUBLIC_APP_URL=http://localhost:3000
NEXT_PUBLIC_SUPABASE_URL=your_supabase_url
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_supabase_anon_key
SUPABASE_SERVICE_ROLE_KEY=your_service_role_key
OPENAI_API_KEY=your_openai_api_key
STRIPE_SECRET_KEY=your_stripe_secret_key
STRIPE_WEBHOOK_SECRET=your_webhook_secret
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=your_publishable_key
STRIPE_PRICE_FREE=price_free
STRIPE_PRICE_PRO=price_pro
STRIPE_PRICE_ENTERPRISE=price_enterprise
```

---

4) app/layout.tsx

```
import './globals.css';
import { AppProviders } from '@/components/providers/AppProviders';
import { Logo } from '@/components/shared/Logo';
```

```
export const metadata = {
  title: 'Superagent Jroc AI',
  description: 'AI tools, automation, verification, and agent marketplace.'
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body className="bg-zinc-950 text-zinc-100">
        <AppProviders>
          <div className="min-h-screen">
            {children}
          </div>
        </AppProviders>
      </body>
    </html>
  );
}
```

---

5) app/globals.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
  color-scheme: dark;
}

body {
  font-family: Arial, Helvetica, sans-serif;
}

.container-app {
  @apply mx-auto max-w-7xl px-4 sm:px-6 lg:px-8;
}

.glass {
  @apply border border-white/10 bg-white/5 backdrop-blur;
}
```

---

6) components/shared/Logo.tsx

```
export function Logo() {
  return (
    <div className="flex items-center gap-3 font-semibold tracking-wide">
      <div className="flex h-10 w-10 items-center justify-center rounded-2xl bg-white
text-black">
        SJ
      </div>
      <div>
        <div className="text-sm text-zinc-400">Superagent</div>
        <div className="text-lg">Jroc AI</div>
      </div>
    </div>
  );
}
```

---

7) app/(marketing)/layout.tsx

```
import Link from 'next/link';
import { Logo } from '@/components/shared/Logo';

export default function MarketingLayout({ children }: { children: React.ReactNode }) {
  return (
    <div>
      <header className="border-b border-white/10">
        <div className="container-app flex h-20 items-center justify-between">
          <Logo />
          <nav className="flex items-center gap-6 text-sm text-zinc-300">
            <Link href="/pricing">Pricing</Link>
            <Link href="/marketplace">Marketplace</Link>
            <Link href="/automation-agents">Agents</Link>
            <Link href="/human-verification">Verification</Link>
            <Link href="/login" className="rounded-xl border border-white/10 px-4
py-2">Login</Link>
            <Link href="/signup" className="rounded-xl bg-white px-4 py-2 text-black">Get
Started</Link>
          </nav>
        </div>
      </header>
      {children}
    </div>
```

```
  );
}
```

---

8) app/(marketing)/page.tsx

```
import Link from 'next/link';
import { ToolMarketplaceGrid } from '@/components/marketing/ToolMarketplaceGrid';
import { PricingCards } from '@/components/marketing/PricingCards';
import { FeatureGrid } from '@/components/marketing/FeatureGrid';

export default function HomePage() {
  return (
    <main>
      <section className="container-app py-20">
        <div className="grid gap-10 lg:grid-cols-2 lg:items-center">
          <div>
            <div className="mb-4 inline-flex rounded-full border border-white/10 px-4 py-2 text-xs
uppercase tracking-[0.25em] text-zinc-400">
              AI SaaS • Human Verification • Agent Automation
            </div>
            <h1 className="max-w-3xl text-5xl font-bold leading-tight sm:text-6xl">
              Build, sell, and automate with <span className="text-zinc-300">Superagent Jroc
AI</span>
            </h1>
            <p className="mt-6 max-w-2xl text-lg text-zinc-400">
              Launch 50 AI tools, run business automations, add human verification, monetize with
subscriptions,
              and scale into a marketplace platform.
            </p>
            <div className="mt-8 flex flex-wrap gap-4">
              <Link href="/signup" className="rounded-2xl bg-white px-6 py-3 text-black">Start
Free</Link>
              <Link href="/marketplace" className="rounded-2xl border border-white/10 px-6
py-3">View Tools</Link>
            </div>
          </div>
          <div className="glass rounded-[2rem] p-6">
            <div className="grid gap-4 sm:grid-cols-2">
              {[
                ['50+', 'Sellable AI Tools'],
                ['10', 'Automation Agents'],
                ['24/7', 'Human Review Layer'],
```

```
            ['Stripe', 'Built-in Billing']
          ].map(([value, label]) => (
            <div key={label} className="rounded-2xl border border-white/10 p-5">
              <div className="text-3xl font-bold">{value}</div>
              <div className="mt-2 text-sm text-zinc-400">{label}</div>
            </div>
          ))}
        </div>
      </div>
    </div>
  </section>
  <FeatureGrid />
  <ToolMarketplaceGrid limit={12} />
  <PricingCards />
</main>
);
}
```

---

9) data/plans.ts

```
export const plans = [
  {
    id: 'free',
    name: 'Free',
    price: '$0',
    description: 'Good for testing the platform.',
    features: ['5 tool runs/day', '1 agent', 'Community verification', 'Basic dashboard']
  },
  {
    id: 'pro',
    name: 'Pro',
    price: '$29/mo',
    description: 'For creators and small teams.',
    features: ['Unlimited tool runs', '10 agents', 'API keys', 'Workflow automation', 'Priority
support']
  },
  {
    id: 'enterprise',
    name: 'Enterprise',
    price: '$99/mo',
    description: 'For agencies and scaled operations.',
```

```
    features: ['Unlimited agents', 'Multi-user team access', 'White-label tools', 'Priority verification', 'Advanced analytics']
  }
];
```

---

10) data/ai-tools.ts — 50 AI Tools Modules

```
export type ToolDefinition = {
  slug: string;
  name: string;
  category: 'content' | 'business' | 'creator' | 'automation' | 'coding' | 'productivity';
  description: string;
  promptTemplate: string;
  priceHint: string;
};

export const aiTools: ToolDefinition[] = [
  { slug: 'blog-writer', name: 'Blog Writer', category: 'content', description: 'Generate full blog posts.', promptTemplate: 'Write a detailed blog post about: {{input}}', priceHint: '$9/mo add-on' },
  { slug: 'seo-optimizer', name: 'SEO Optimizer', category: 'content', description: 'Improve content for search.', promptTemplate: 'Optimize this for SEO: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'email-writer', name: 'Email Writer', category: 'content', description: 'Write outreach and campaigns.', promptTemplate: 'Write a polished email for: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'ad-copy-generator', name: 'Ad Copy Generator', category: 'content', description: 'Create ads for Meta and Google.', promptTemplate: 'Create 10 high-converting ads for: {{input}}', priceHint: '$12/mo add-on' },
  { slug: 'product-description', name: 'Product Description Writer', category: 'content', description: 'Turn rough product notes into conversion-ready copy.', promptTemplate: 'Write a compelling product description for: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'press-release', name: 'Press Release Writer', category: 'content', description: 'Professional PR drafts.', promptTemplate: 'Write a press release for: {{input}}', priceHint: '$7/mo add-on' },
  { slug: 'social-posts', name: 'Social Post Generator', category: 'content', description: 'Generate platform-specific social posts.', promptTemplate: 'Create 15 social posts for: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'youtube-titles', name: 'YouTube Title Generator', category: 'creator', description: 'Generate titles and hooks.', promptTemplate: 'Create 20 YouTube titles for: {{input}}', priceHint: '$5/mo add-on' },
  { slug: 'video-script', name: 'Video Script Writer', category: 'creator', description: 'Generate short and long-form scripts.', promptTemplate: 'Write a video script for: {{input}}', priceHint: '$15/mo add-on' },
```

```
  { slug: 'podcast-outline', name: 'Podcast Outline Builder', category: 'creator', description: 'Plan episodes and talking points.', promptTemplate: 'Create a podcast outline about: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'logo-brief', name: 'Logo Brief Generator', category: 'creator', description: 'Creative direction for brand marks.', promptTemplate: 'Create a logo design brief for: {{input}}', priceHint: '$10/mo add-on' },
  { slug: 'brand-voice', name: 'Brand Voice Builder', category: 'creator', description: 'Define tone and messaging.', promptTemplate: 'Define a brand voice guide for: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'sales-script', name: 'Sales Script Writer', category: 'business', description: 'Cold calls and objection handling.', promptTemplate: 'Write a sales script for: {{input}}', priceHint: '$19/mo add-on' },
  { slug: 'proposal-builder', name: 'Proposal Builder', category: 'business', description: 'Client proposal drafts.', promptTemplate: 'Write a client proposal for: {{input}}', priceHint: '$12/mo add-on' },
  { slug: 'crm-notes', name: 'CRM Notes Summarizer', category: 'business', description: 'Summarize meetings and deal context.', promptTemplate: 'Summarize these CRM notes: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'lead-scoring', name: 'Lead Scoring AI', category: 'business', description: 'Score and prioritize leads.', promptTemplate: 'Score and prioritize these leads: {{input}}', priceHint: '$29/mo add-on' },
  { slug: 'support-reply', name: 'Support Reply Assistant', category: 'business', description: 'Draft fast support responses.', promptTemplate: 'Draft a customer support reply for: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'faq-builder', name: 'FAQ Builder', category: 'business', description: 'Generate help center content.', promptTemplate: 'Generate an FAQ for: {{input}}', priceHint: '$9/mo add-on' },
  { slug: 'resume-builder', name: 'Resume Builder', category: 'productivity', description: 'Create professional resumes.', promptTemplate: 'Write a resume based on: {{input}}', priceHint: '$8/mo add-on' },
  { slug: 'cover-letter', name: 'Cover Letter Writer', category: 'productivity', description: 'Tailored applications.', promptTemplate: 'Write a cover letter for: {{input}}', priceHint: '$6/mo add-on' },
  { slug: 'research-assistant', name: 'Research Assistant', category: 'productivity', description: 'Analyze topics and summarize findings.', promptTemplate: 'Research and summarize: {{input}}', priceHint: '$24/mo add-on' },
  { slug: 'meeting-summary', name: 'Meeting Summarizer', category: 'productivity', description: 'Extract actions and takeaways.', promptTemplate: 'Summarize this meeting transcript: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'idea-generator', name: 'Idea Generator', category: 'productivity', description: 'Generate startup and campaign ideas.', promptTemplate: 'Generate 25 ideas for: {{input}}', priceHint: '$5/mo add-on' },
  { slug: 'translator', name: 'Translator', category: 'productivity', description: 'Translate with context.', promptTemplate: 'Translate this text preserving tone: {{input}}', priceHint: 'Included in Pro' },
```

```
  { slug: 'summarizer', name: 'Summarizer', category: 'productivity', description: 'Compress long text.', promptTemplate: 'Summarize the following text: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'code-generator', name: 'Code Generator', category: 'coding', description: 'Generate code from prompts.', promptTemplate: 'Write production-ready code for: {{input}}', priceHint: '$29/mo add-on' },
  { slug: 'bug-fixer', name: 'Bug Fix Assistant', category: 'coding', description: 'Debug code and explain issues.', promptTemplate: 'Fix and explain this bug: {{input}}', priceHint: '$19/mo add-on' },
  { slug: 'api-docs', name: 'API Docs Generator', category: 'coding', description: 'Generate developer docs.', promptTemplate: 'Write API documentation for: {{input}}', priceHint: '$9/mo add-on' },
  { slug: 'sql-builder', name: 'SQL Query Builder', category: 'coding', description: 'Generate SQL safely.', promptTemplate: 'Write an SQL query for: {{input}}', priceHint: '$12/mo add-on' },
  { slug: 'landing-page-copy', name: 'Landing Page Copy', category: 'coding', description: 'Conversion copy for launches.', promptTemplate: 'Write landing page copy for: {{input}}', priceHint: '$14/mo add-on' },
  { slug: 'website-generator', name: 'Website Generator', category: 'coding', description: 'Generate website structure and sections.', promptTemplate: 'Create website structure and content for: {{input}}', priceHint: '$39/mo add-on' },
  { slug: 'form-builder', name: 'Form Builder Assistant', category: 'coding', description: 'Generate forms and validation logic.', promptTemplate: 'Generate a web form and validation for: {{input}}', priceHint: '$11/mo add-on' },
  { slug: 'workflow-builder', name: 'Workflow Builder', category: 'automation', description: 'Create automation flows.', promptTemplate: 'Design an automation workflow for: {{input}}', priceHint: '$25/mo add-on' },
  { slug: 'email-automation', name: 'Email Automation AI', category: 'automation', description: 'Sequences and triggers.', promptTemplate: 'Create an email automation flow for: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'lead-gen-automation', name: 'Lead Gen Automation', category: 'automation', description: 'Design outbound flows.', promptTemplate: 'Build a lead generation automation for: {{input}}', priceHint: '$29/mo add-on' },
  { slug: 'customer-support-agent', name: 'Customer Support Agent', category: 'automation', description: 'Self-serve support workflows.', promptTemplate: 'Build a support automation for: {{input}}', priceHint: '$35/mo add-on' },
  { slug: 'sales-agent', name: 'Sales Agent', category: 'automation', description: 'Pipeline follow-up automation.', promptTemplate: 'Build a sales automation for: {{input}}', priceHint: '$35/mo add-on' },
  { slug: 'marketing-agent', name: 'Marketing Agent', category: 'automation', description: 'Campaign orchestration.', promptTemplate: 'Build a marketing automation for: {{input}}', priceHint: '$35/mo add-on' },
  { slug: 'social-scheduler', name: 'Social Scheduler', category: 'automation', description: 'Plan and automate posting.', promptTemplate: 'Create a social posting schedule for: {{input}}', priceHint: '$19/mo add-on' },
```

```
  { slug: 'seo-cluster-builder', name: 'SEO Cluster Builder', category: 'content', description: 'Topic clusters and internal linking.', promptTemplate: 'Create an SEO content cluster for: {{input}}', priceHint: '$19/mo add-on' },
  { slug: 'competitor-analysis', name: 'Competitor Analysis', category: 'business', description: 'Compare offers and positioning.', promptTemplate: 'Analyze competitors for: {{input}}', priceHint: '$22/mo add-on' },
  { slug: 'pricing-strategy', name: 'Pricing Strategy Generator', category: 'business', description: 'Model offers and plans.', promptTemplate: 'Create a pricing strategy for: {{input}}', priceHint: '$18/mo add-on' },
  { slug: 'onboarding-flow', name: 'Onboarding Flow Designer', category: 'business', description: 'Improve activation rates.', promptTemplate: 'Design an onboarding flow for: {{input}}', priceHint: '$16/mo add-on' },
  { slug: 'survey-analyzer', name: 'Survey Analyzer', category: 'productivity', description: 'Analyze user feedback.', promptTemplate: 'Analyze this survey data: {{input}}', priceHint: '$14/mo add-on' },
  { slug: 'knowledge-base-writer', name: 'Knowledge Base Writer', category: 'business', description: 'Help docs and internal SOPs.', promptTemplate: 'Write a knowledge base article for: {{input}}', priceHint: '$10/mo add-on' },
  { slug: 'headline-generator', name: 'Headline Generator', category: 'content', description: 'Generate headlines and hooks.', promptTemplate: 'Generate 50 headlines for: {{input}}', priceHint: '$7/mo add-on' },
  { slug: 'cold-dm-writer', name: 'Cold DM Writer', category: 'business', description: 'Draft outreach messages.', promptTemplate: 'Write cold DMs for: {{input}}', priceHint: '$9/mo add-on' },
  { slug: 'script-rewriter', name: 'Script Rewriter', category: 'creator', description: 'Improve clarity and pacing.', promptTemplate: 'Rewrite this script for better performance: {{input}}', priceHint: '$13/mo add-on' },
  { slug: 'ugc-brief-generator', name: 'UGC Brief Generator', category: 'creator', description: 'Creator campaign briefs.', promptTemplate: 'Write a UGC creator brief for: {{input}}', priceHint: '$11/mo add-on' },
  { slug: 'micro-saas-idea-generator', name: 'Micro SaaS Idea Generator', category: 'productivity', description: 'Generate startup concepts.', promptTemplate: 'Generate profitable micro-SaaS ideas around: {{input}}', priceHint: '$17/mo add-on' },
  { slug: 'agent-marketplace-listing', name: 'Agent Marketplace Listing Writer', category: 'automation', description: 'Write marketplace-ready listings.', promptTemplate: 'Write an agent marketplace listing for: {{input}}', priceHint: '$8/mo add-on' }
];
```

---

11) components/marketing/ToolMarketplaceGrid.tsx

```
import { aiTools } from '@/data/ai-tools';
```

```
export function ToolMarketplaceGrid({ limit }: { limit?: number }) {
  const items = typeof limit === 'number' ? aiTools.slice(0, limit) : aiTools;
  return (
    <section className="container-app py-20">
      <div className="mb-8">
        <h2 className="text-3xl font-bold">50 AI Tools You Can Sell</h2>
        <p className="mt-2 text-zinc-400">A modular marketplace powered by one shared tool
runner API.</p>
      </div>
      <div className="grid gap-4 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4">
        {items.map((tool) => (
          <div key={tool.slug} className="glass rounded-3xl p-5">
            <div className="text-xs uppercase tracking-[0.2em]
text-zinc-500">{tool.category}</div>
            <h3 className="mt-3 text-lg font-semibold">{tool.name}</h3>
            <p className="mt-2 text-sm text-zinc-400">{tool.description}</p>
            <div className="mt-4 text-sm text-zinc-300">{tool.priceHint}</div>
          </div>
        ))}
      </div>
    </section>
  );
}
```

---

12) app/(marketing)/marketplace/page.tsx

```
import { ToolMarketplaceGrid } from '@/components/marketing/ToolMarketplaceGrid';

export default function MarketplacePage() {
  return <ToolMarketplaceGrid />;
}
```

---

13) components/marketing/PricingCards.tsx

```
import Link from 'next/link';
import { plans } from '@/data/plans';

export function PricingCards() {
  return (
    <section className="container-app py-20">
```

```
      <div className="mb-8">
        <h2 className="text-3xl font-bold">Simple SaaS Pricing</h2>
        <p className="mt-2 text-zinc-400">Free, Pro, and Enterprise with Stripe checkout.</p>
      </div>
      <div className="grid gap-6 lg:grid-cols-3">
        {plans.map((plan) => (
          <div key={plan.id} className="glass rounded-[2rem] p-6">
            <div className="text-sm uppercase tracking-[0.25em]
text-zinc-500">{plan.name}</div>
            <div className="mt-4 text-4xl font-bold">{plan.price}</div>
            <p className="mt-3 text-zinc-400">{plan.description}</p>
            <ul className="mt-6 space-y-3 text-sm text-zinc-300">
              {plan.features.map((f) => <li key={f}>• {f}</li>)}
            </ul>
            <Link href="/signup" className="mt-6 inline-flex rounded-2xl bg-white px-5 py-3
text-black">
              Choose {plan.name}
            </Link>
          </div>
        ))}
      </div>
    </section>
  );
}
```

---

14) app/(marketing)/pricing/page.tsx

```
import { PricingCards } from '@/components/marketing/PricingCards';

export default function PricingPage() {
  return <PricingCards />;
}
```

---

15) app/(auth)/login/page.tsx

```
export default function LoginPage() {
  return (
    <main className="container-app py-20">
      <div className="mx-auto max-w-md rounded-[2rem] border border-white/10 bg-white/5
p-8">
```

```
        <h1 className="text-3xl font-bold">Login</h1>
        <form className="mt-6 space-y-4">
          <input className="w-full rounded-2xl border border-white/10 bg-zinc-900 px-4 py-3"
placeholder="Email" />
          <input className="w-full rounded-2xl border border-white/10 bg-zinc-900 px-4 py-3"
placeholder="Password" type="password" />
          <button className="w-full rounded-2xl bg-white px-4 py-3 text-black">Sign In</button>
        </form>
      </div>
    </main>
  );
}
```

---

16) app/(auth)/signup/page.tsx

```
export default function SignupPage() {
  return (
    <main className="container-app py-20">
      <div className="mx-auto max-w-md rounded-[2rem] border border-white/10 bg-white/5
p-8">
        <h1 className="text-3xl font-bold">Create Account</h1>
        <form className="mt-6 space-y-4">
          <input className="w-full rounded-2xl border border-white/10 bg-zinc-900 px-4 py-3"
placeholder="Full name" />
          <input className="w-full rounded-2xl border border-white/10 bg-zinc-900 px-4 py-3"
placeholder="Email" />
          <input className="w-full rounded-2xl border border-white/10 bg-zinc-900 px-4 py-3"
placeholder="Password" type="password" />
          <button className="w-full rounded-2xl bg-white px-4 py-3 text-black">Start
Free</button>
        </form>
      </div>
    </main>
  );
}
```

---

17) components/dashboard/Sidebar.tsx

```
import Link from 'next/link';
```

```
const items = [
  ['Overview', '/dashboard'],
  ['Tools', '/dashboard/tools'],
  ['Agents', '/dashboard/agents'],
  ['Automation', '/dashboard/automation'],
  ['Verification', '/dashboard/verification'],
  ['Billing', '/dashboard/billing'],
  ['API Keys', '/dashboard/api-keys'],
  ['Settings', '/dashboard/settings']
];

export function Sidebar() {
  return (
    <aside className="hidden w-72 shrink-0 border-r border-white/10 p-6 lg:block">
      <div className="mb-8 text-xl font-bold">Dashboard</div>
      <nav className="space-y-2">
        {items.map(([label, href]) => (
          <Link key={href} href={href} className="block rounded-2xl px-4 py-3 text-zinc-300
hover:bg-white/5">
            {label}
          </Link>
        ))}
      </nav>
    </aside>
  );
}
```

---

18) components/dashboard/DashboardShell.tsx

```
import { Sidebar } from './Sidebar';

export function DashboardShell({ title, children }: { title: string; children: React.ReactNode }) {
  return (
    <div className="min-h-screen lg:flex">
      <Sidebar />
      <main className="flex-1 p-6 lg:p-10">
        <div className="mb-8 flex items-center justify-between">
          <h1 className="text-3xl font-bold">{title}</h1>
          <button className="rounded-2xl bg-white px-4 py-2 text-black">Upgrade</button>
        </div>
        {children}
      </main>
```

```
    </div>
  );
}
```

---

19) app/dashboard/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

export default function DashboardPage() {
  return (
    <DashboardShell title="Overview">
      <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-4">
        {[
          ['Total Tool Runs', '12,480'],
          ['Agents Active', '8'],
          ['Verification Jobs', '126'],
          ['MRR Potential', '$18,240']
        ].map(([label, value]) => (
          <div key={label} className="glass rounded-3xl p-6">
            <div className="text-sm text-zinc-400">{label}</div>
            <div className="mt-2 text-3xl font-bold">{value}</div>
          </div>
        ))}
      </div>
    </DashboardShell>
  );
}
```

---

20) components/dashboard/ToolRunner.tsx

```
'use client';

import { useState } from 'react';
import { aiTools } from '@/data/ai-tools';

export function ToolRunner() {
  const [toolSlug, setToolSlug] = useState(aiTools[0].slug);
  const [input, setInput] = useState('');
  const [output, setOutput] = useState('');
  const [loading, setLoading] = useState(false);
```

```
  async function runTool() {
    setLoading(true);
    setOutput('');
    const res = await fetch('/api/tools/run', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ toolSlug, input })
    });
    const data = await res.json();
    setOutput(data.output || 'No output');
    setLoading(false);
  }

  return (
    <div className="grid gap-6 lg:grid-cols-[320px,1fr]">
      <div className="glass rounded-3xl p-5">
        <label className="mb-2 block text-sm text-zinc-400">Choose Tool</label>
        <select className="w-full rounded-2xl bg-zinc-900 p-3" value={toolSlug} onChange={(e)
=> setToolSlug(e.target.value)}>
          {aiTools.map((tool) => <option key={tool.slug} value={tool.slug}>{tool.name}</option>)}
        </select>
        <textarea
          className="mt-4 min-h-[220px] w-full rounded-2xl bg-zinc-900 p-3"
          placeholder="Describe what you want the AI to do"
          value={input}
          onChange={(e) => setInput(e.target.value)}
        />
        <button onClick={runTool} className="mt-4 w-full rounded-2xl bg-white px-4 py-3
text-black">
          {loading ? 'Running...' : 'Run Tool'}
        </button>
      </div>
      <div className="glass rounded-3xl p-5">
        <div className="mb-3 text-sm text-zinc-400">Output</div>
        <pre className="whitespace-pre-wrap text-sm text-zinc-200">{output}</pre>
      </div>
    </div>
  );
}
```

---

21) app/dashboard/tools/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';
import { ToolRunner } from '@/components/dashboard/ToolRunner';

export default function DashboardToolsPage() {
  return (
    <DashboardShell title="AI Tools">
      <ToolRunner />
    </DashboardShell>
  );
}
```

---

22) data/agent-templates.ts

```
export const agentTemplates = [
  { slug: 'support-agent', name: 'Customer Support Agent', role: 'Resolve FAQs, triage tickets,
and draft replies.' },
  { slug: 'marketing-agent', name: 'Marketing Agent', role: 'Generate campaigns, hooks, and
promotional calendars.' },
  { slug: 'sales-agent', name: 'Sales Agent', role: 'Follow up with leads, score deals, and create
scripts.' },
  { slug: 'lead-agent', name: 'Lead Generation Agent', role: 'Identify prospects, qualify leads, and
suggest outreach.' },
  { slug: 'email-agent', name: 'Email Automation Agent', role: 'Build sequences and run drip
campaigns.' },
  { slug: 'social-agent', name: 'Social Media Agent', role: 'Schedule and generate social content
at scale.' },
  { slug: 'site-agent', name: 'Website Builder Agent', role: 'Generate sitemap, copy, and CTA
strategy.' },
  { slug: 'research-agent', name: 'Research Agent', role: 'Analyze industries, trends, and
positioning.' },
  { slug: 'ops-agent', name: 'Operations Agent', role: 'Generate SOPs, forms, and internal
workflows.' },
  { slug: 'growth-agent', name: 'Growth Hacking Agent', role: 'Test offers, channels, and
experiments.' }
];
```

---

23) components/dashboard/AgentBuilder.tsx

```
'use client';

import { useState } from 'react';
import { agentTemplates } from '@/data/agent-templates';

export function AgentBuilder() {
  const [name, setName] = useState('My Revenue Agent');
  const [template, setTemplate] = useState(agentTemplates[0].slug);
  const [goal, setGoal] = useState('Help convert more leads into sales.');
  const [result, setResult] = useState('');

  async function createAgent() {
    const res = await fetch('/api/agents/create', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name, template, goal })
    });
    const data = await res.json();
    setResult(JSON.stringify(data, null, 2));
  }

  return (
    <div className="grid gap-6 lg:grid-cols-[360px,1fr]">
      <div className="glass rounded-3xl p-5">
        <label className="mb-2 block text-sm text-zinc-400">Agent Name</label>
        <input className="w-full rounded-2xl bg-zinc-900 p-3" value={name} onChange={(e) =>
setName(e.target.value)} />
        <label className="mb-2 mt-4 block text-sm text-zinc-400">Template</label>
        <select className="w-full rounded-2xl bg-zinc-900 p-3" value={template} onChange={(e)
=> setTemplate(e.target.value)}>
          {agentTemplates.map((t) => <option key={t.slug} value={t.slug}>{t.name}</option>)}
        </select>
        <label className="mb-2 mt-4 block text-sm text-zinc-400">Goal</label>
        <textarea className="min-h-[180px] w-full rounded-2xl bg-zinc-900 p-3" value={goal}
onChange={(e) => setGoal(e.target.value)} />
        <button onClick={createAgent} className="mt-4 w-full rounded-2xl bg-white px-4 py-3
text-black">Create Agent</button>
      </div>
      <div className="glass rounded-3xl p-5">
        <div className="mb-3 text-sm text-zinc-400">Agent JSON Output</div>
        <pre className="whitespace-pre-wrap text-sm text-zinc-200">{result}</pre>
      </div>
    </div>
  );
```

```
}
```

---

24) app/dashboard/agents/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';
import { AgentBuilder } from '@/components/dashboard/AgentBuilder';

export default function DashboardAgentsPage() {
  return (
    <DashboardShell title="AI Agent Builder">
      <AgentBuilder />
    </DashboardShell>
  );
}
```

---

25) components/dashboard/VerificationJobForm.tsx

```
'use client';

import { useState } from 'react';

export function VerificationJobForm() {
  const [category, setCategory] = useState('fact-checking');
  const [content, setContent] = useState('');
  const [response, setResponse] = useState('');

  async function submitJob() {
    const res = await fetch('/api/verification/jobs', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ category, content })
    });
    const data = await res.json();
    setResponse(`Job created: ${data.jobId}`);
  }

  return (
    <div className="glass rounded-3xl p-5">
      <h2 className="text-xl font-semibold">Submit Verification Job</h2>
```

```
      <select className="mt-4 w-full rounded-2xl bg-zinc-900 p-3" value={category}
onChange={(e) => setCategory(e.target.value)}>
        {['content-moderation', 'fact-checking', 'spam-detection', 'quality-control', 'image-review',
'video-moderation', 'ai-hallucination-detection'].map((c) => (
          <option key={c} value={c}>{c}</option>
        ))}
      </select>
      <textarea
        className="mt-4 min-h-[220px] w-full rounded-2xl bg-zinc-900 p-3"
        value={content}
        onChange={(e) => setContent(e.target.value)}
        placeholder="Paste the AI output or content to verify"
      />
      <button onClick={submitJob} className="mt-4 rounded-2xl bg-white px-4 py-3
text-black">Submit for Review</button>
      <div className="mt-4 text-sm text-zinc-400">{response}</div>
    </div>
  );
}
```

---

26) components/dashboard/VerificationMarketplace.tsx

```
export function VerificationMarketplace() {
  const workers = [
    { name: 'Moderator One', rating: 4.9, jobs: 1200, specialty: 'Content Moderation' },
    { name: 'Checker Pro', rating: 4.8, jobs: 860, specialty: 'Fact Checking' },
    { name: 'Vision Reviewer', rating: 4.7, jobs: 510, specialty: 'Image Review' }
  ];

  return (
    <div className="glass rounded-3xl p-5">
      <h2 className="text-xl font-semibold">Human Verification Marketplace</h2>
      <div className="mt-5 space-y-4">
        {workers.map((worker) => (
          <div key={worker.name} className="rounded-2xl border border-white/10 p-4">
            <div className="flex flex-wrap items-center justify-between gap-3">
              <div>
                <div className="font-semibold">{worker.name}</div>
                <div className="text-sm text-zinc-400">{worker.specialty}</div>
              </div>
              <div className="text-sm text-zinc-300">⭐ {worker.rating} • {worker.jobs} jobs</div>
            </div>
```

```
        </div>
      ))}
    </div>
  </div>
  );
}
```

---

27) app/dashboard/verification/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';
import { VerificationJobForm } from '@/components/dashboard/VerificationJobForm';
import { VerificationMarketplace } from '@/components/dashboard/VerificationMarketplace';

export default function DashboardVerificationPage() {
  return (
    <DashboardShell title="Human Verification">
      <div className="grid gap-6 lg:grid-cols-2">
        <VerificationJobForm />
        <VerificationMarketplace />
      </div>
    </DashboardShell>
  );
}
```

---

28) app/(marketing)/human-verification/page.tsx

```
export default function HumanVerificationPage() {
  return (
    <main className="container-app py-20">
      <div className="max-w-4xl">
        <h1 className="text-5xl font-bold">Human Verification Marketplace</h1>
        <p className="mt-6 text-lg text-zinc-400">
          Add a human-in-the-loop layer to fact-check, moderate, score quality, and review AI
outputs before they reach customers.
        </p>
      </div>
      <div className="mt-10 grid gap-4 md:grid-cols-2 xl:grid-cols-4">
        {['Fact Checking', 'Spam Detection', 'Quality Control', 'Image Review'].map((item) => (
          <div key={item} className="glass rounded-3xl p-5">
            <div className="text-xl font-semibold">{item}</div>
```

```
          <p className="mt-2 text-sm text-zinc-400">Marketplace-ready workflow with jobs,
worker ratings, and review queues.</p>
        </div>
      ))}
    </div>
  </main>
 );
}
```

---

29) app/(marketing)/automation-agents/page.tsx

```
import { agentTemplates } from '@/data/agent-templates';

export default function AutomationAgentsPage() {
 return (
  <main className="container-app py-20">
   <h1 className="text-5xl font-bold">Automation Agents</h1>
   <p className="mt-4 max-w-2xl text-lg text-zinc-400">
    Deploy agents for sales, support, growth, operations, and content workflows.
   </p>
   <div className="mt-10 grid gap-4 md:grid-cols-2 xl:grid-cols-3">
    {agentTemplates.map((agent) => (
     <div key={agent.slug} className="glass rounded-3xl p-5">
      <h2 className="text-xl font-semibold">{agent.name}</h2>
      <p className="mt-2 text-sm text-zinc-400">{agent.role}</p>
     </div>
    ))}
   </div>
  </main>
 );
}
```

---

30) lib/openai.ts

```
import OpenAI from 'openai';
import { env } from './env';

export const openai = new OpenAI({
 apiKey: env.OPENAI_API_KEY
});
```

---

31) lib/env.ts

```
function getEnv(name: string, fallback?: string) {
  const value = process.env[name] || fallback;
  if (!value) throw new Error(`Missing environment variable: ${name}`);
  return value;
}

export const env = {
  NEXT_PUBLIC_APP_URL: getEnv('NEXT_PUBLIC_APP_URL'),
  NEXT_PUBLIC_SUPABASE_URL: getEnv('NEXT_PUBLIC_SUPABASE_URL'),
  NEXT_PUBLIC_SUPABASE_ANON_KEY:
getEnv('NEXT_PUBLIC_SUPABASE_ANON_KEY'),
  SUPABASE_SERVICE_ROLE_KEY: getEnv('SUPABASE_SERVICE_ROLE_KEY'),
  OPENAI_API_KEY: getEnv('OPENAI_API_KEY'),
  STRIPE_SECRET_KEY: getEnv('STRIPE_SECRET_KEY'),
  STRIPE_WEBHOOK_SECRET: getEnv('STRIPE_WEBHOOK_SECRET')
};
```

---

32) lib/supabase-server.ts

```
import { createClient } from '@supabase/supabase-js';
import { env } from './env';

export function createAdminSupabaseClient() {
  return createClient(env.NEXT_PUBLIC_SUPABASE_URL,
env.SUPABASE_SERVICE_ROLE_KEY);
}
```

---

33) lib/supabase-client.ts

```
import { createClient } from '@supabase/supabase-js';
import { env } from './env';

export const supabaseBrowser = createClient(
  env.NEXT_PUBLIC_SUPABASE_URL,
  env.NEXT_PUBLIC_SUPABASE_ANON_KEY
```

```
);
```

---

34) lib/stripe.ts

```
import Stripe from 'stripe';
import { env } from './env';

export const stripe = new Stripe(env.STRIPE_SECRET_KEY, {
  apiVersion: '2025-02-24.acacia'
});
```

---

35) app/api/tools/run/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { aiTools } from '@/data/ai-tools';
import { openai } from '@/lib/openai';

export async function POST(request: NextRequest) {
  try {
    const { toolSlug, input } = await request.json();
    const tool = aiTools.find((t) => t.slug === toolSlug);

    if (!tool) {
      return NextResponse.json({ error: 'Tool not found' }, { status: 404 });
    }

    const prompt = tool.promptTemplate.replace('{{input}}', input || '');

    const completion = await openai.responses.create({
      model: 'gpt-4.1-mini',
      input: prompt
    });

    const output = completion.output_text || 'No output generated.';

    return NextResponse.json({ success: true, tool: tool.name, output });
  } catch (error) {
    return NextResponse.json({ error: 'Failed to run tool' }, { status: 500 });
  }
}
```

---

36) app/api/agents/create/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { agentTemplates } from '@/data/agent-templates';

export async function POST(request: NextRequest) {
  const body = await request.json();
  const template = agentTemplates.find((t) => t.slug === body.template);

  return NextResponse.json({
    success: true,
    agent: {
      id: crypto.randomUUID(),
      name: body.name,
      template: body.template,
      goal: body.goal,
      config: {
        role: template?.role,
        triggers: ['manual', 'schedule', 'webhook'],
        actions: ['generate', 'analyze', 'reply', 'summarize']
      }
    }
  });
}
```

---

37) app/api/agents/run/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { openai } from '@/lib/openai';

export async function POST(request: NextRequest) {
  const { agentName, goal, context } = await request.json();

  const prompt = `You are ${agentName}. Your goal is: ${goal}. Use this context: ${context}`;

  const completion = await openai.responses.create({
    model: 'gpt-4.1-mini',
    input: prompt
  });
```

```
  return NextResponse.json({
    success: true,
    output: completion.output_text
  });
}
```

---

38) app/api/verification/jobs/route.ts

```
import { NextRequest, NextResponse } from 'next/server';

export async function POST(request: NextRequest) {
  const { category, content } = await request.json();

  return NextResponse.json({
    success: true,
    jobId: crypto.randomUUID(),
    category,
    status: 'queued',
    preview: String(content || '').slice(0, 160)
  });
}
```

---

39) app/api/verification/worker-apply/route.ts

```
import { NextRequest, NextResponse } from 'next/server';

export async function POST(request: NextRequest) {
  const { name, email, specialty } = await request.json();

  return NextResponse.json({
    success: true,
    applicationId: crypto.randomUUID(),
    name,
    email,
    specialty,
    status: 'pending_review'
  });
}
```

---

40) app/api/billing/checkout/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { stripe } from '@/lib/stripe';

export async function POST(request: NextRequest) {
  const { priceId } = await request.json();

  const session = await stripe.checkout.sessions.create({
    mode: 'subscription',
    line_items: [{ price: priceId, quantity: 1 }],
    success_url: `${process.env.NEXT_PUBLIC_APP_URL}/dashboard/billing?success=true`,
    cancel_url: `${process.env.NEXT_PUBLIC_APP_URL}/pricing?canceled=true`
  });

  return NextResponse.json({ url: session.url });
}
```

---

41) app/api/billing/portal/route.ts

```
import { NextResponse } from 'next/server';

export async function POST() {
  return NextResponse.json({
    success: true,
    message: 'Attach Stripe customer portal here once customer IDs are persisted.'
  });
}
```

---

42) app/api/stripe/webhook/route.ts

```
import { NextRequest, NextResponse } from 'next/server';

export async function POST(_request: NextRequest) {
  return NextResponse.json({ received: true });
}
```

---

43) app/dashboard/billing/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';
import { plans } from '@/data/plans';

export default function BillingPage() {
  return (
    <DashboardShell title="Billing">
      <div className="grid gap-6 lg:grid-cols-3">
        {plans.map((plan) => (
          <div key={plan.id} className="glass rounded-3xl p-6">
            <div className="text-xl font-semibold">{plan.name}</div>
            <div className="mt-2 text-4xl font-bold">{plan.price}</div>
            <ul className="mt-4 space-y-2 text-sm text-zinc-300">
              {plan.features.map((feature) => <li key={feature}>• {feature}</li>)}
            </ul>
            <button className="mt-6 rounded-2xl bg-white px-4 py-3
text-black">Checkout</button>
          </div>
        ))}
      </div>
    </DashboardShell>
  );
}
```

---

44) app/dashboard/api-keys/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

export default function ApiKeysPage() {
  return (
    <DashboardShell title="API Keys">
      <div className="glass rounded-3xl p-6">
        <h2 className="text-xl font-semibold">Developer Access</h2>
        <p className="mt-2 text-zinc-400">Generate API keys so customers can embed
Superagent Jroc AI into other apps.</p>
        <div className="mt-6 flex gap-4">
          <button className="rounded-2xl bg-white px-4 py-3 text-black">Generate Key</button>
          <div className="rounded-2xl border border-white/10 px-4 py-3 text-sm
text-zinc-300">sk_live_jroc_xxxxxxxxx</div>
        </div>
      </div>
```

```
    </DashboardShell>
  );
}
```

---

45) app/dashboard/settings/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

export default function SettingsPage() {
  return (
    <DashboardShell title="Settings">
      <div className="glass rounded-3xl p-6">
        <h2 className="text-xl font-semibold">Account Settings</h2>
        <div className="mt-6 grid gap-4 md:grid-cols-2">
          <input className="rounded-2xl bg-zinc-900 p-3" placeholder="Business Name" />
          <input className="rounded-2xl bg-zinc-900 p-3" placeholder="Website" />
          <input className="rounded-2xl bg-zinc-900 p-3" placeholder="Support Email" />
          <input className="rounded-2xl bg-zinc-900 p-3" placeholder="Timezone" />
        </div>
        <button className="mt-6 rounded-2xl bg-white px-4 py-3 text-black">Save
Changes</button>
      </div>
    </DashboardShell>
  );
}
```

---

46) app/admin/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

export default function AdminPage() {
  return (
    <DashboardShell title="Admin Panel">
      <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-4">
        {[
          ['Users', '4,208'],
          ['MRR', '$32,940'],
          ['Workers', '314'],
          ['Jobs Today', '1,482']
        ].map(([label, value]) => (
```

```
        <div key={label} className="glass rounded-3xl p-6">
          <div className="text-sm text-zinc-400">{label}</div>
          <div className="mt-2 text-3xl font-bold">{value}</div>
        </div>
      ))}
    </div>
  </DashboardShell>
 );
}
```

---

47) db/schema.sql

```
create extension if not exists pgcrypto;

create table if not exists profiles (
  id uuid primary key default gen_random_uuid(),
  email text unique not null,
  full_name text,
  role text default 'user',
  created_at timestamptz default now()
);

create table if not exists subscriptions (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id) on delete cascade,
  stripe_customer_id text,
  stripe_subscription_id text,
  plan text not null default 'free',
  status text default 'inactive',
  created_at timestamptz default now()
);

create table if not exists api_keys (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id) on delete cascade,
  key_hash text not null,
  label text,
  last_used_at timestamptz,
  created_at timestamptz default now()
);

create table if not exists usage_logs (
```

```
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id) on delete cascade,
  tool_slug text,
  run_type text,
  units integer default 1,
  metadata jsonb default '{}'::jsonb,
  created_at timestamptz default now()
);

create table if not exists agents (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id) on delete cascade,
  name text not null,
  template_slug text,
  goal text,
  config jsonb default '{}'::jsonb,
  status text default 'draft',
  created_at timestamptz default now()
);

create table if not exists automation_runs (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id) on delete cascade,
  agent_id uuid references agents(id) on delete cascade,
  status text default 'queued',
  input jsonb default '{}'::jsonb,
  output jsonb default '{}'::jsonb,
  created_at timestamptz default now()
);

create table if not exists verification_workers (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id) on delete cascade,
  specialty text,
  rating numeric(3,2) default 5.0,
  total_jobs integer default 0,
  status text default 'pending',
  created_at timestamptz default now()
);

create table if not exists verification_jobs (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id) on delete cascade,
  category text not null,
```

```
  content text not null,
  status text default 'queued',
  assigned_worker_id uuid references verification_workers(id),
  result jsonb default '{}'::jsonb,
  created_at timestamptz default now()
);

create table if not exists payouts (
  id uuid primary key default gen_random_uuid(),
  worker_id uuid references verification_workers(id) on delete cascade,
  amount_cents integer not null,
  status text default 'pending',
  created_at timestamptz default now()
);
```

---

48) README.md

```
# Superagent Jroc AI

A modern AI SaaS platform with:
- 50 AI tools marketplace
- Agent builder
- Human verification marketplace
- Stripe subscriptions
- API keys
- Admin dashboard

## Local Setup

1. Clone repo
2. Copy `.env.example` to `.env.local`
3. Add Supabase, Stripe, and OpenAI keys
4. Install packages:
   ```bash
   npm install

Run dev:
 npm run dev

      5.
```

Deploy
```

- Frontend: Vercel
- Database/Auth: Supabase
- Billing: Stripe

Monetization

- SaaS subscriptions
- Tool add-ons
- Agent marketplace commissions
- Verification job fees
- API usage revenue

---

49) $10M Startup Architecture

Core Revenue Engines
1. **Subscriptions** — Free / Pro / Enterprise
2. **Add-on tools** — Sell premium tools by category
3. **Verification fees** — Charge per human-reviewed job
4. **Agent marketplace** — Take commission on community agents
5. **API access** — Usage-based pricing for developers
6. **Enterprise onboarding** — White-label and setup fees

Growth Flywheel
- Users join for one tool
- Upgrade for workflows
- Add agents to save labor
- Add human review for trust
- Invite workers and creators
- Launch agent marketplace
- Sell API to businesses

Moat
- AI + human verification
- Multi-tool bundle under one login
- Marketplace network effects
- Usage data for improving templates
- Enterprise workflow lock-in

12-Month Expansion
- Team accounts
- Affiliate/referral system

- White-label dashboards
- Agent marketplace with reviews
- Vertical packs: real estate, ecommerce, agencies, creators

$10M Path Example
- 3,000 Pro users at $29/mo = $87,000 MRR
- 500 Enterprise users at $99/mo = $49,500 MRR
- Add-ons + API + marketplace + verification = $100k–$250k MRR upside

---

50) What To Build Next

Highest-impact next files
1. Supabase auth flow with protected routes
2. Stripe webhook persistence
3. Real user billing state in dashboard
4. Agent run logs and scheduler
5. Worker payout logic
6. Admin moderation queue
7. Affiliate/referral system
8. White-label settings

Best scaling decision
Use **one universal tool runner** with the `aiTools` config file rather than hand-coding 50 separate backends. That gives you 50 sellable modules with much less maintenance.

---

51) Copy-Paste Prompt for GitHub Copilot

```text
Build the Superagent Jroc AI repo exactly as specified in this document.
Use Next.js 14 App Router, TypeScript, Tailwind, Supabase, Stripe, and OpenAI.
Keep the dark premium dashboard styling.
Implement all routes and components listed.
Use the aiTools config file as the source of truth for the 50 tool modules.
Use a single /api/tools/run endpoint that loads prompt templates dynamically by slug.
Create production-safe code with reusable components, clean types, and organized folders.
Add protected dashboard routes, a working Stripe checkout flow, and Supabase-ready schema integration.
```

---

52) Phase 2 — Real Auth, Route Protection, Billing Wiring, and Deploy

Below are the next files to make the starter repo feel much more real.

middleware.ts

```
import { NextResponse, type NextRequest } from 'next/server';

const protectedPaths = ['/dashboard', '/admin'];

export function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;
  const needsAuth = protectedPaths.some((path) => pathname.startsWith(path));

  const session = request.cookies.get('sjroc_session')?.value;

  if (needsAuth && !session) {
    const url = request.nextUrl.clone();
    url.pathname = '/login';
    return NextResponse.redirect(url);
  }

  return NextResponse.next();
}

export const config = {
  matcher: ['/dashboard/:path*', '/admin/:path*']
};
```

lib/auth.ts

```
import { cookies } from 'next/headers';

export async function getSessionUser() {
  const cookieStore = await cookies();
  const raw = cookieStore.get('sjroc_session')?.value;
  if (!raw) return null;

  try {
    return JSON.parse(raw);
  } catch {
    return null;
```

```
  }
}

export async function requireUser() {
  const user = await getSessionUser();
  if (!user) {
    throw new Error('Unauthorized');
  }
  return user;
}
```

app/api/auth/session/route.ts

```
import { NextRequest, NextResponse } from 'next/server';

export async function POST(request: NextRequest) {
  const { email, password } = await request.json();

  if (!email || !password) {
    return NextResponse.json({ error: 'Missing credentials' }, { status: 400 });
  }

  const response = NextResponse.json({
    success: true,
    user: {
      id: crypto.randomUUID(),
      email,
      role: email.includes('admin') ? 'admin' : 'user'
    }
  });

  response.cookies.set(
    'sjroc_session',
    JSON.stringify({
      id: crypto.randomUUID(),
      email,
      role: email.includes('admin') ? 'admin' : 'user'
    }),
    {
      httpOnly: true,
      sameSite: 'lax',
      secure: process.env.NODE_ENV === 'production',
      path: '/'
    }
```

```
  );

  return response;
}

export async function DELETE() {
  const response = NextResponse.json({ success: true });
  response.cookies.set('sjroc_session', '', {
    httpOnly: true,
    expires: new Date(0),
    path: '/'
  });
  return response;
}
```

app/(auth)/login/page.tsx

```
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';

export default function LoginPage() {
  const router = useRouter();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  async function handleLogin(e: React.FormEvent) {
    e.preventDefault();
    setLoading(true);
    setError('');

    const res = await fetch('/api/auth/session', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password })
    });

    const data = await res.json();

    if (!res.ok) {
      setError(data.error || 'Login failed');
```

```
      setLoading(false);
      return;
    }

    router.push('/dashboard');
    router.refresh();
  }

  return (
    <main className="container-app py-20">
      <div className="mx-auto max-w-md rounded-[2rem] border border-white/10 bg-white/5
p-8">
        <h1 className="text-3xl font-bold">Login</h1>
        <form className="mt-6 space-y-4" onSubmit={handleLogin}>
          <input
            className="w-full rounded-2xl border border-white/10 bg-zinc-900 px-4 py-3"
            placeholder="Email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
          />
          <input
            className="w-full rounded-2xl border border-white/10 bg-zinc-900 px-4 py-3"
            placeholder="Password"
            type="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
          />
          {error ? <div className="text-sm text-red-400">{error}</div> : null}
          <button className="w-full rounded-2xl bg-white px-4 py-3 text-black">
            {loading ? 'Signing in...' : 'Sign In'}
          </button>
        </form>
      </div>
    </main>
  );
}
```

app/api/health/route.ts

```
import { NextResponse } from 'next/server';

export async function GET() {
  return NextResponse.json({
    ok: true,
```

```
    service: 'Superagent Jroc AI',
    timestamp: new Date().toISOString()
  });
}
```

lib/pricing.ts

```
export const STRIPE_PRICE_MAP = {
  free: process.env.STRIPE_PRICE_FREE || 'price_free',
  pro: process.env.STRIPE_PRICE_PRO || 'price_pro',
  enterprise: process.env.STRIPE_PRICE_ENTERPRISE || 'price_enterprise'
};
```

app/dashboard/billing/page.tsx

```
'use client';

import { DashboardShell } from '@/components/dashboard/DashboardShell';
import { plans } from '@/data/plans';
import { STRIPE_PRICE_MAP } from '@/lib/pricing';

export default function BillingPage() {
  async function startCheckout(planId: 'free' | 'pro' | 'enterprise') {
    if (planId === 'free') return;

    const res = await fetch('/api/billing/checkout', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ priceId: STRIPE_PRICE_MAP[planId] })
    });

    const data = await res.json();
    if (data.url) {
      window.location.href = data.url;
    }
  }

  return (
    <DashboardShell title="Billing">
      <div className="grid gap-6 lg:grid-cols-3">
        {plans.map((plan) => (
          <div key={plan.id} className="glass rounded-3xl p-6">
            <div className="text-xl font-semibold">{plan.name}</div>
```

```
          <div className="mt-2 text-4xl font-bold">{plan.price}</div>
          <ul className="mt-4 space-y-2 text-sm text-zinc-300">
            {plan.features.map((feature) => <li key={feature}>• {feature}</li>)}
          </ul>
          <button
            onClick={() => startCheckout(plan.id as 'free' | 'pro' | 'enterprise')}
            className="mt-6 rounded-2xl bg-white px-4 py-3 text-black"
          >
            {plan.id === 'free' ? 'Current Plan' : `Upgrade to ${plan.name}`}
          </button>
        </div>
      ))}
    </div>
  </DashboardShell>
);
}
```

app/api/stripe/webhook/route.ts

```
import { headers } from 'next/headers';
import { NextRequest, NextResponse } from 'next/server';
import { stripe } from '@/lib/stripe';

export async function POST(request: NextRequest) {
  const body = await request.text();
  const signature = (await headers()).get('stripe-signature');

  if (!signature) {
    return NextResponse.json({ error: 'Missing stripe signature' }, { status: 400 });
  }

  try {
    const event = stripe.webhooks.constructEvent(
      body,
      signature,
      process.env.STRIPE_WEBHOOK_SECRET as string
    );

    switch (event.type) {
      case 'checkout.session.completed':
        break;
      case 'customer.subscription.updated':
        break;
      case 'customer.subscription.deleted':
```

```
      break;
    default:
      break;
  }

  return NextResponse.json({ received: true, type: event.type });
 } catch (error) {
  return NextResponse.json({ error: 'Invalid webhook' }, { status: 400 });
 }
}
```

app/dashboard/automation/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

const flows = [
 {
  name: 'Lead Follow-up Flow',
  trigger: 'New inbound lead',
  action: 'Score lead, write intro email, queue human review'
 },
 {
  name: 'Content Approval Flow',
  trigger: 'Blog post generated',
  action: 'Run SEO tool, summarize changes, submit verification job'
 },
 {
  name: 'Client Onboarding Flow',
  trigger: 'Stripe checkout completed',
  action: 'Create workspace, provision API key, start onboarding emails'
 }
];

export default function AutomationPage() {
 return (
  <DashboardShell title="Automation Workflows">
   <div className="grid gap-4 xl:grid-cols-3">
    {flows.map((flow) => (
     <div key={flow.name} className="glass rounded-3xl p-6">
      <div className="text-xl font-semibold">{flow.name}</div>
      <div className="mt-4 text-sm text-zinc-400">Trigger</div>
      <div className="mt-1 text-zinc-200">{flow.trigger}</div>
      <div className="mt-4 text-sm text-zinc-400">Actions</div>
      <div className="mt-1 text-zinc-200">{flow.action}</div>
```

```
        </div>
      ))}
    </div>
  </DashboardShell>
 );
}
```

app/admin/users/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

const users = [
 { email: 'owner@superagentjroc.ai', plan: 'Enterprise', status: 'active' },
 { email: 'growth@client.com', plan: 'Pro', status: 'active' },
 { email: 'creator@brand.com', plan: 'Free', status: 'trial' }
];

export default function AdminUsersPage() {
 return (
  <DashboardShell title="Admin Users">
   <div className="glass rounded-3xl overflow-hidden">
    <table className="w-full text-left text-sm">
     <thead className="border-b border-white/10 text-zinc-400">
      <tr>
       <th className="p-4">Email</th>
       <th className="p-4">Plan</th>
       <th className="p-4">Status</th>
      </tr>
     </thead>
     <tbody>
      {users.map((user) => (
       <tr key={user.email} className="border-b border-white/5">
        <td className="p-4">{user.email}</td>
        <td className="p-4">{user.plan}</td>
        <td className="p-4">{user.status}</td>
       </tr>
      ))}
     </tbody>
    </table>
   </div>
  </DashboardShell>
 );
}
```

app/admin/payments/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

const payments = [
  { id: 'pay_001', email: 'owner@superagentjroc.ai', amount: '$99.00', status: 'paid' },
  { id: 'pay_002', email: 'growth@client.com', amount: '$29.00', status: 'paid' },
  { id: 'pay_003', email: 'trial@brand.com', amount: '$0.00', status: 'free' }
];

export default function AdminPaymentsPage() {
  return (
    <DashboardShell title="Admin Payments">
      <div className="grid gap-4">
        {payments.map((payment) => (
          <div key={payment.id} className="glass rounded-3xl p-5 flex items-center
justify-between gap-4">
            <div>
              <div className="font-semibold">{payment.email}</div>
              <div className="text-sm text-zinc-400">{payment.id}</div>
            </div>
            <div className="text-zinc-200">{payment.amount}</div>
            <div className="text-sm text-zinc-400">{payment.status}</div>
          </div>
        ))}
      </div>
    </DashboardShell>
  );
}
```

app/admin/workers/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

const workers = [
  { name: 'Moderator One', specialty: 'Content Moderation', rating: 4.9, status: 'approved' },
  { name: 'Checker Pro', specialty: 'Fact Checking', rating: 4.8, status: 'approved' },
  { name: 'New Reviewer', specialty: 'Image Review', rating: 5.0, status: 'pending' }
];

export default function AdminWorkersPage() {
  return (
    <DashboardShell title="Admin Workers">
      <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-3">
```

```
      {workers.map((worker) => (
        <div key={worker.name} className="glass rounded-3xl p-6">
          <div className="text-xl font-semibold">{worker.name}</div>
          <div className="mt-2 text-sm text-zinc-400">{worker.specialty}</div>
          <div className="mt-4 text-zinc-200">Rating: {worker.rating}</div>
          <div className="mt-2 text-zinc-400">Status: {worker.status}</div>
        </div>
      ))}
    </div>
  </DashboardShell>
);
}
```

app/admin/analytics/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

const metrics = [
  ['MRR', '$132,400'],
  ['Tool Runs', '280,442'],
  ['Verification Jobs', '14,201'],
  ['Avg Revenue per User', '$41.70'],
  ['Churn', '3.1%'],
  ['Marketplace GMV', '$58,340']
];

export default function AdminAnalyticsPage() {
  return (
    <DashboardShell title="Admin Analytics">
      <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-3">
        {metrics.map(([label, value]) => (
          <div key={label} className="glass rounded-3xl p-6">
            <div className="text-sm text-zinc-400">{label}</div>
            <div className="mt-2 text-3xl font-bold">{value}</div>
          </div>
        ))}
      </div>
    </DashboardShell>
  );
}
```

Deploy Steps

```
npm install
npm run dev
```

Then deploy:

```
vercel
```

What To Build Immediately After This

1. Replace cookie mock auth with real Supabase auth.
2. Persist Stripe customer IDs and plan status in Supabase.
3. Save agent configs and tool runs in the database.
4. Add worker payouts and review queues.
5. Add usage limits by plan.
6. Add affiliate links and referral tracking.
7. Add white-label custom branding for Enterprise.

---

53) Exact Next Prompt To Paste Into GitHub Copilot

Continue the Superagent Jroc AI repo.
Now implement real Supabase authentication, protected dashboard sessions, Stripe webhook subscription syncing, saved tool run history, saved agent records, verification worker applications, admin approval actions, and plan-based usage limits.
Keep the same premium dark SaaS design and preserve the existing file structure.

---

54) GitHub Codespaces Setup

Add these files so the repo opens and runs correctly in **GitHub Codespaces**.

.devcontainer/devcontainer.json

```
{
  "name": "Superagent Jroc AI Codespace",
  "image": "mcr.microsoft.com/devcontainers/javascript-node:1-20-bookworm",
  "customizations": {
    "vscode": {
      "settings": {
        "terminal.integrated.defaultProfile.linux": "bash",
        "editor.formatOnSave": true,
```

```
      "files.eol": "
"
    },
    "extensions": [
      "dbaeumer.vscode-eslint",
      "esbenp.prettier-vscode",
      "bradlc.vscode-tailwindcss",
      "ms-vscode.vscode-typescript-next",
      "GitHub.copilot",
      "GitHub.copilot-chat"
    ]
  }
},
"forwardPorts": [3000],
"portsAttributes": {
  "3000": {
    "label": "Next.js App",
    "onAutoForward": "openPreview"
  }
},
"postCreateCommand": "npm install && cp .env.example .env.local || true",
"remoteUser": "node"
}
```

.gitignore

```
node_modules
.next
.env.local
.env
.vercel
coverage
*.log
```

next.config.mjs

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    serverActions: {
      bodySizeLimit: '2mb'
    }
  }
```

```
};

export default nextConfig;
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["dom", "dom.iterable", "es2022"],
    "allowJs": false,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "baseUrl": ".",
    "paths": {
      "@/*": ["./*"]
    },
    "plugins": [{ "name": "next" ]
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules"]
}
```

Fix for tsconfig.json plugin line

Use this corrected version instead if needed:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["dom", "dom.iterable", "es2022"],
    "allowJs": false,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
```

```
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "baseUrl": ".",
    "paths": {
      "@/*": ["./*"]
    },
    "plugins": [{ "name": "next" }]
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules"]
}
```

postcss.config.js

```
module.exports = {
  plugins: {
    tailwindcss: {},
    autoprefixer: {}
  }
};
```

tailwind.config.ts

```
import type { Config } from 'tailwindcss';

const config: Config = {
  content: [
    './app/**/*.{js,ts,jsx,tsx,mdx}',
    './components/**/*.{js,ts,jsx,tsx,mdx}',
    './pages/**/*.{js,ts,jsx,tsx,mdx}'
  ],
  theme: {
    extend: {}
  },
  plugins: []
};

export default config;
```

next-env.d.ts

```
/// <reference types="next" />
/// <reference types="next/image-types/global" />

// NOTE: This file should not be edited.
```

.eslintrc.json

```
{
  "extends": ["next/core-web-vitals"]
}
```

app/api/health/route.ts

```
import { NextResponse } from 'next/server';

export async function GET() {
  return NextResponse.json({
    ok: true,
    service: 'Superagent Jroc AI',
    timestamp: new Date().toISOString()
  });
}
```

package.json

Use this safer Codespaces-ready version:

```
{
  "name": "superagent-jroc-ai",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev -H 0.0.0.0 -p 3000",
    "build": "next build",
    "start": "next start -H 0.0.0.0 -p 3000",
    "lint": "next lint"
  },
  "dependencies": {
    "@supabase/ssr": "latest",
```

```
    "@supabase/supabase-js": "latest",
    "lucide-react": "latest",
    "next": "14.2.30",
    "openai": "latest",
    "react": "18.3.1",
    "react-dom": "18.3.1",
    "stripe": "latest",
    "zod": "latest"
  },
  "devDependencies": {
    "@types/node": "latest",
    "@types/react": "latest",
    "@types/react-dom": "latest",
    "autoprefixer": "latest",
    "eslint": "latest",
    "eslint-config-next": "14.2.30",
    "postcss": "latest",
    "tailwindcss": "latest",
    "typescript": "latest"
  }
}
```

Terminal commands for Codespaces

```
npm install
cp .env.example .env.local
npm run dev
```

Then open the forwarded port for 3000.

---

55) Fast Copy-Paste Prompt for GitHub Copilot in Codespaces

Set up this Superagent Jroc AI repo to run perfectly in GitHub Codespaces.
Create .devcontainer/devcontainer.json, next.config.mjs, tsconfig.json, tailwind.config.ts, postcss.config.js, next-env.d.ts, .eslintrc.json, and update package.json so the app runs on port 3000 with host 0.0.0.0.
Make sure npm install works cleanly and the Next.js app launches inside Codespaces with forwarded port 3000.

---

56) Exact Folder To Create For Codespaces

```
superagent-jroc-ai/
  .devcontainer/
    devcontainer.json
  app/
  components/
  data/
  lib/
  db/
  public/
  types/
  .env.example
  .eslintrc.json
  .gitignore
  next-env.d.ts
  next.config.mjs
  package.json
  postcss.config.js
  tailwind.config.ts
  tsconfig.json
```

Added the **next phase** into the canvas:

- protected dashboard/admin routes
- login session API
- billing checkout wiring
- Stripe webhook starter
- automation page
- admin users/payments/workers/analytics pages
- exact next Copilot prompt

Your next move is:

1. Copy these new files into GitHub
2. Run the repo locally
3. Paste the new Copilot prompt to expand real auth + database persistence

Use this prompt next in Copilot:

Continue the Superagent Jroc AI repo.
Now implement real Supabase authentication, protected dashboard sessions, Stripe webhook subscription syncing, saved tool run history, saved agent records, verification worker applications, admin approval actions, and plan-based usage limits.

Keep the same premium dark SaaS design and preserve the existing file structure.

Say **Next** again and I'll generate the **real Supabase auth + Stripe sync + database save layer**.

Added the **next phase** into the canvas:

- protected dashboard/admin routes
- login session API
- billing checkout wiring
- Stripe webhook starter
- automation page
- admin users/payments/workers/analytics pages
- exact next Copilot prompt

Your next move is:

1. Copy these new files into GitHub
2. Run the repo locally
3. Paste the new Copilot prompt to expand real auth + database persistence

Use this prompt next in Copilot:

Continue the Superagent Jroc AI repo.
Now implement real Supabase authentication, protected dashboard sessions, Stripe webhook subscription syncing, saved tool run history, saved agent records, verification worker applications, admin approval actions, and plan-based usage limits.
Keep the same premium dark SaaS design and preserve the existing file structure.

Say **Next** again and I'll generate the **real Supabase auth + Stripe sync + database save layer**.

Alright — the **next step** is giving you **ONE FILE you can paste into GitHub Codespaces that auto-generates the entire Superagent Jroc AI project**. This is the easiest way to import everything at once. 🚀

You will create **one setup script**.

1️⃣ Create This File In Your Repo

Create a file called:

setup-superagent.sh

Paste this entire code:

```
#!/bin/bash

echo "🚀 Creating Superagent Jroc AI SaaS Platform..."

# Create folders
mkdir -p superagent-jroc-ai
cd superagent-jroc-ai

mkdir -p app/api/tools
mkdir -p app/api/agents
mkdir -p app/api/verification
mkdir -p app/dashboard
mkdir -p components
mkdir -p data
mkdir -p lib
mkdir -p styles
mkdir -p public

# Package.json
cat > package.json << 'EOF'
{
  "name": "superagent-jroc-ai",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev -H 0.0.0.0 -p 3000",
    "build": "next build",
    "start": "next start"
  },
  "dependencies": {
    "next": "14.2.30",
    "react": "18.3.1",
    "react-dom": "18.3.1",
    "openai": "latest",
    "@supabase/supabase-js": "latest",
    "stripe": "latest",
    "tailwindcss": "latest"
  }
}
EOF
```

```
# Next config
cat > next.config.mjs << 'EOF'
const nextConfig = {};
export default nextConfig;
EOF

# Homepage
mkdir -p app

cat > app/page.tsx << 'EOF'
export default function Home() {
  return (
    <main style={{padding:"40px"}}>
      <h1>Superagent Jroc AI</h1>
      <p>AI Tools • Automation Agents • Human Verification Marketplace</p>
    </main>
  )
}
EOF

# Dashboard
mkdir -p app/dashboard

cat > app/dashboard/page.tsx << 'EOF'
export default function Dashboard() {
  return (
    <div style={{padding:"40px"}}>
      <h2>Superagent Dashboard</h2>
      <p>Run AI tools, build agents, manage verification jobs.</p>
    </div>
  )
}
EOF

# AI tools list
mkdir -p data

cat > data/tools.js << 'EOF'
export const tools = [
 {name:"Blog Writer"},
 {name:"SEO Optimizer"},
 {name:"Email Writer"},
 {name:"Ad Copy Generator"},
```

```
 {name:"AI Sales Assistant"},
 {name:"Resume Builder"},
 {name:"AI Translator"},
 {name:"Research Assistant"},
 {name:"AI Coding Assistant"},
 {name:"Landing Page Builder"}
]
EOF

# OpenAI client
mkdir -p lib

cat > lib/openai.js << 'EOF'
import OpenAI from "openai"

export const openai = new OpenAI({
 apiKey: process.env.OPENAI_API_KEY
})
EOF

# AI tool API
mkdir -p app/api/tools

cat > app/api/tools/run.js << 'EOF'
import { openai } from "../../../lib/openai"

export async function POST(req) {

 const body = await req.json()

 const completion = await openai.chat.completions.create({
  model: "gpt-4.1-mini",
  messages:[
   {role:"system",content:"You are an AI SaaS tool."},
   {role:"user",content:body.prompt}
  ]
 })

 return Response.json({
  output: completion.choices[0].message.content
 })
}
EOF
```

```
# Human verification API
mkdir -p app/api/verification

cat > app/api/verification/job.js << 'EOF'
export async function POST(req){

 const data = await req.json()

 return Response.json({
  status:"queued",
  job:data
 })

}
EOF

# Agent builder API
mkdir -p app/api/agents

cat > app/api/agents/create.js << 'EOF'
export async function POST(req){

 const data = await req.json()

 return Response.json({
  agent:{
   id:Math.random(),
   name:data.name,
   goal:data.goal
  }
 })

}
EOF

echo "✅ Superagent Jroc AI project created!"
echo "Run: npm install"
echo "Then: npm run dev"
```

2️⃣ Run It In Codespaces

In the terminal:

```
chmod +x setup-superagent.sh
./setup-superagent.sh
```

Then run:

```
npm install
npm run dev
```

Open **port 3000**.

3️⃣ What You Will See

Your first working version of:

Superagent Jroc AI

With:

✅ homepage
✅ dashboard
✅ AI tool API
✅ agent builder API
✅ human verification API
✅ tool modules

4️⃣ After This (Next Step)

I can generate the **ULTIMATE VERSION** that includes:

- **50 AI tools fully wired**
- **real Stripe SaaS payments**
- **Supabase login system**
- **AI agent builder UI**
- **human verification worker marketplace**

- **AI automation workflow builder**
- **AI marketplace where users sell agents**

This is the version that can realistically become a **$10M SaaS platform**.

Just say:

“Build the Superagent Jroc AI $10M version.” 🚀

Superagent Jroc AI — Production-Grade Starter Repo

Below is a complete multi-file **starter repository** for **Superagent Jroc AI**: a multi-page AI SaaS platform with auth, billing, agent builder, human verification marketplace, admin dashboard, and 50 sellable AI tools.

Stack: **Next.js 14 App Router**, **TypeScript**, **Tailwind CSS**, **Supabase**, **Stripe**, **OpenAI**

---

1) Repo Tree

```
superagent-jroc-ai/
├─ app/
│  ├─ (marketing)/
│  │  ├─ page.tsx
│  │  ├─ pricing/page.tsx
│  │  ├─ marketplace/page.tsx
│  │  ├─ automation-agents/page.tsx
│  │  ├─ human-verification/page.tsx
│  │  └─ layout.tsx
│  ├─ (auth)/
│  │  ├─ login/page.tsx
│  │  └─ signup/page.tsx
│  ├─ dashboard/
│  │  ├─ page.tsx
│  │  ├─ tools/page.tsx
│  │  ├─ agents/page.tsx
│  │  ├─ automation/page.tsx
│  │  ├─ verification/page.tsx
│  │  ├─ billing/page.tsx
│  │  ├─ settings/page.tsx
```

```
│   │   └─ api-keys/page.tsx
│   ├─ admin/
│   │   ├─ page.tsx
│   │   ├─ users/page.tsx
│   │   ├─ payments/page.tsx
│   │   ├─ workers/page.tsx
│   │   └─ analytics/page.tsx
│   ├─ api/
│   │   ├─ tools/run/route.ts
│   │   ├─ agents/create/route.ts
│   │   ├─ agents/run/route.ts
│   │   ├─ verification/jobs/route.ts
│   │   ├─ verification/worker-apply/route.ts
│   │   ├─ billing/checkout/route.ts
│   │   ├─ billing/portal/route.ts
│   │   ├─ stripe/webhook/route.ts
│   │   ├─ auth/session/route.ts
│   │   └─ health/route.ts
│   ├─ globals.css
│   └─ layout.tsx
├─ components/
│   ├─ marketing/
│   │   ├─ Hero.tsx
│   │   ├─ FeatureGrid.tsx
│   │   ├─ PricingCards.tsx
│   │   └─ ToolMarketplaceGrid.tsx
│   ├─ dashboard/
│   │   ├─ DashboardShell.tsx
│   │   ├─ Sidebar.tsx
│   │   ├─ Topbar.tsx
│   │   ├─ StatCard.tsx
│   │   ├─ ToolRunner.tsx
│   │   ├─ AgentBuilder.tsx
│   │   ├─ AutomationCanvas.tsx
│   │   ├─ VerificationJobForm.tsx
│   │   ├─ VerificationMarketplace.tsx
│   │   └─ BillingPanel.tsx
│   ├─ shared/
│   │   ├─ Logo.tsx
│   │   ├─ Button.tsx
│   │   ├─ Card.tsx
│   │   ├─ Input.tsx
│   │   └─ SectionHeading.tsx
│   └─ providers/
```

```
│     └─ AppProviders.tsx
├─ lib/
│  ├─ env.ts
│  ├─ supabase-server.ts
│  ├─ supabase-client.ts
│  ├─ auth.ts
│  ├─ stripe.ts
│  ├─ openai.ts
│  ├─ usage.ts
│  ├─ pricing.ts
│  ├─ permissions.ts
│  ├─ utils.ts
│  └─ validators.ts
├─ data/
│  ├─ ai-tools.ts
│  ├─ agent-templates.ts
│  └─ plans.ts
├─ db/
│  ├─ schema.sql
│  └─ seed.sql
├─ public/
│  └─ favicon.ico
├─ types/
│  ├─ tool.ts
│  ├─ agent.ts
│  ├─ billing.ts
│  └─ verification.ts
├─ middleware.ts
├─ package.json
├─ tsconfig.json
├─ tailwind.config.ts
├─ postcss.config.js
├─ next.config.mjs
├─ .env.example
└─ README.md
```

---

2) package.json

```
{
  "name": "superagent-jroc-ai",
  "version": "1.0.0",
  "private": true,
```

```
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint"
  },
  "dependencies": {
    "@supabase/ssr": "latest",
    "@supabase/supabase-js": "latest",
    "lucide-react": "latest",
    "next": "14.2.30",
    "openai": "latest",
    "react": "18.3.1",
    "react-dom": "18.3.1",
    "stripe": "latest",
    "zod": "latest"
  },
  "devDependencies": {
    "@types/node": "latest",
    "@types/react": "latest",
    "@types/react-dom": "latest",
    "autoprefixer": "latest",
    "postcss": "latest",
    "tailwindcss": "latest",
    "typescript": "latest"
  }
}
```

---

3) .env.example

```
NEXT_PUBLIC_APP_URL=http://localhost:3000
NEXT_PUBLIC_SUPABASE_URL=your_supabase_url
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_supabase_anon_key
SUPABASE_SERVICE_ROLE_KEY=your_service_role_key
OPENAI_API_KEY=your_openai_api_key
STRIPE_SECRET_KEY=your_stripe_secret_key
STRIPE_WEBHOOK_SECRET=your_webhook_secret
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=your_publishable_key
STRIPE_PRICE_FREE=price_free
STRIPE_PRICE_PRO=price_pro
STRIPE_PRICE_ENTERPRISE=price_enterprise
```

---

4) app/layout.tsx

```
import './globals.css';
import { AppProviders } from '@/components/providers/AppProviders';
import { Logo } from '@/components/shared/Logo';

export const metadata = {
  title: 'Superagent Jroc AI',
  description: 'AI tools, automation, verification, and agent marketplace.'
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body className="bg-zinc-950 text-zinc-100">
        <AppProviders>
          <div className="min-h-screen">
            {children}
          </div>
        </AppProviders>
      </body>
    </html>
  );
}
```

---

5) app/globals.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
  color-scheme: dark;
}

body {
  font-family: Arial, Helvetica, sans-serif;
}

.container-app {
```

```
  @apply mx-auto max-w-7xl px-4 sm:px-6 lg:px-8;
}

.glass {
  @apply border border-white/10 bg-white/5 backdrop-blur;
}
```

---

6) components/shared/Logo.tsx

```
export function Logo() {
  return (
    <div className="flex items-center gap-3 font-semibold tracking-wide">
      <div className="flex h-10 w-10 items-center justify-center rounded-2xl bg-white
text-black">
        SJ
      </div>
      <div>
        <div className="text-sm text-zinc-400">Superagent</div>
        <div className="text-lg">Jroc AI</div>
      </div>
    </div>
  );
}
```

---

7) app/(marketing)/layout.tsx

```
import Link from 'next/link';
import { Logo } from '@/components/shared/Logo';

export default function MarketingLayout({ children }: { children: React.ReactNode }) {
  return (
    <div>
      <header className="border-b border-white/10">
        <div className="container-app flex h-20 items-center justify-between">
          <Logo />
          <nav className="flex items-center gap-6 text-sm text-zinc-300">
            <Link href="/pricing">Pricing</Link>
            <Link href="/marketplace">Marketplace</Link>
            <Link href="/automation-agents">Agents</Link>
            <Link href="/human-verification">Verification</Link>
```

```
          <Link href="/login" className="rounded-xl border border-white/10 px-4
py-2">Login</Link>
          <Link href="/signup" className="rounded-xl bg-white px-4 py-2 text-black">Get
Started</Link>
        </nav>
      </div>
    </header>
    {children}
  </div>
 );
}
```

---

8) app/(marketing)/page.tsx

```
import Link from 'next/link';
import { ToolMarketplaceGrid } from '@/components/marketing/ToolMarketplaceGrid';
import { PricingCards } from '@/components/marketing/PricingCards';
import { FeatureGrid } from '@/components/marketing/FeatureGrid';

export default function HomePage() {
 return (
  <main>
    <section className="container-app py-20">
      <div className="grid gap-10 lg:grid-cols-2 lg:items-center">
        <div>
          <div className="mb-4 inline-flex rounded-full border border-white/10 px-4 py-2 text-xs
uppercase tracking-[0.25em] text-zinc-400">
            AI SaaS • Human Verification • Agent Automation
          </div>
          <h1 className="max-w-3xl text-5xl font-bold leading-tight sm:text-6xl">
            Build, sell, and automate with <span className="text-zinc-300">Superagent Jroc
AI</span>
          </h1>
          <p className="mt-6 max-w-2xl text-lg text-zinc-400">
            Launch 50 AI tools, run business automations, add human verification, monetize with
subscriptions,
            and scale into a marketplace platform.
          </p>
          <div className="mt-8 flex flex-wrap gap-4">
            <Link href="/signup" className="rounded-2xl bg-white px-6 py-3 text-black">Start
Free</Link>
```

```
            <Link href="/marketplace" className="rounded-2xl border border-white/10 px-6
py-3">View Tools</Link>
          </div>
        </div>
        <div className="glass rounded-[2rem] p-6">
          <div className="grid gap-4 sm:grid-cols-2">
            {[
              ['50+', 'Sellable AI Tools'],
              ['10', 'Automation Agents'],
              ['24/7', 'Human Review Layer'],
              ['Stripe', 'Built-in Billing']
            ].map(([value, label]) => (
              <div key={label} className="rounded-2xl border border-white/10 p-5">
                <div className="text-3xl font-bold">{value}</div>
                <div className="mt-2 text-sm text-zinc-400">{label}</div>
              </div>
            ))}
          </div>
        </div>
      </div>
    </section>
    <FeatureGrid />
    <ToolMarketplaceGrid limit={12} />
    <PricingCards />
  </main>
  );
}
```

---

9) data/plans.ts

```
export const plans = [
  {
    id: 'free',
    name: 'Free',
    price: '$0',
    description: 'Good for testing the platform.',
    features: ['5 tool runs/day', '1 agent', 'Community verification', 'Basic dashboard']
  },
  {
    id: 'pro',
    name: 'Pro',
    price: '$29/mo',
```

```
    description: 'For creators and small teams.',
    features: ['Unlimited tool runs', '10 agents', 'API keys', 'Workflow automation', 'Priority support']
  },
  {
    id: 'enterprise',
    name: 'Enterprise',
    price: '$99/mo',
    description: 'For agencies and scaled operations.',
    features: ['Unlimited agents', 'Multi-user team access', 'White-label tools', 'Priority verification', 'Advanced analytics']
  }
];
```

---

10) data/ai-tools.ts — 50 AI Tools Modules

```
export type ToolDefinition = {
  slug: string;
  name: string;
  category: 'content' | 'business' | 'creator' | 'automation' | 'coding' | 'productivity';
  description: string;
  promptTemplate: string;
  priceHint: string;
};

export const aiTools: ToolDefinition[] = [
  { slug: 'blog-writer', name: 'Blog Writer', category: 'content', description: 'Generate full blog posts.', promptTemplate: 'Write a detailed blog post about: {{input}}', priceHint: '$9/mo add-on' },
  { slug: 'seo-optimizer', name: 'SEO Optimizer', category: 'content', description: 'Improve content for search.', promptTemplate: 'Optimize this for SEO: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'email-writer', name: 'Email Writer', category: 'content', description: 'Write outreach and campaigns.', promptTemplate: 'Write a polished email for: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'ad-copy-generator', name: 'Ad Copy Generator', category: 'content', description: 'Create ads for Meta and Google.', promptTemplate: 'Create 10 high-converting ads for: {{input}}', priceHint: '$12/mo add-on' },
  { slug: 'product-description', name: 'Product Description Writer', category: 'content', description: 'Turn rough product notes into conversion-ready copy.', promptTemplate: 'Write a compelling product description for: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'press-release', name: 'Press Release Writer', category: 'content', description: 'Professional PR drafts.', promptTemplate: 'Write a press release for: {{input}}', priceHint: '$7/mo add-on' },
```

```
  { slug: 'social-posts', name: 'Social Post Generator', category: 'content', description: 'Generate platform-specific social posts.', promptTemplate: 'Create 15 social posts for: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'youtube-titles', name: 'YouTube Title Generator', category: 'creator', description: 'Generate titles and hooks.', promptTemplate: 'Create 20 YouTube titles for: {{input}}', priceHint: '$5/mo add-on' },
  { slug: 'video-script', name: 'Video Script Writer', category: 'creator', description: 'Generate short and long-form scripts.', promptTemplate: 'Write a video script for: {{input}}', priceHint: '$15/mo add-on' },
  { slug: 'podcast-outline', name: 'Podcast Outline Builder', category: 'creator', description: 'Plan episodes and talking points.', promptTemplate: 'Create a podcast outline about: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'logo-brief', name: 'Logo Brief Generator', category: 'creator', description: 'Creative direction for brand marks.', promptTemplate: 'Create a logo design brief for: {{input}}', priceHint: '$10/mo add-on' },
  { slug: 'brand-voice', name: 'Brand Voice Builder', category: 'creator', description: 'Define tone and messaging.', promptTemplate: 'Define a brand voice guide for: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'sales-script', name: 'Sales Script Writer', category: 'business', description: 'Cold calls and objection handling.', promptTemplate: 'Write a sales script for: {{input}}', priceHint: '$19/mo add-on' },
  { slug: 'proposal-builder', name: 'Proposal Builder', category: 'business', description: 'Client proposal drafts.', promptTemplate: 'Write a client proposal for: {{input}}', priceHint: '$12/mo add-on' },
  { slug: 'crm-notes', name: 'CRM Notes Summarizer', category: 'business', description: 'Summarize meetings and deal context.', promptTemplate: 'Summarize these CRM notes: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'lead-scoring', name: 'Lead Scoring AI', category: 'business', description: 'Score and prioritize leads.', promptTemplate: 'Score and prioritize these leads: {{input}}', priceHint: '$29/mo add-on' },
  { slug: 'support-reply', name: 'Support Reply Assistant', category: 'business', description: 'Draft fast support responses.', promptTemplate: 'Draft a customer support reply for: {{input}}', priceHint: 'Included in Pro' },
  { slug: 'faq-builder', name: 'FAQ Builder', category: 'business', description: 'Generate help center content.', promptTemplate: 'Generate an FAQ for: {{input}}', priceHint: '$9/mo add-on' },
  { slug: 'resume-builder', name: 'Resume Builder', category: 'productivity', description: 'Create professional resumes.', promptTemplate: 'Write a resume based on: {{input}}', priceHint: '$8/mo add-on' },
  { slug: 'cover-letter', name: 'Cover Letter Writer', category: 'productivity', description: 'Tailored applications.', promptTemplate: 'Write a cover letter for: {{input}}', priceHint: '$6/mo add-on' },
  { slug: 'research-assistant', name: 'Research Assistant', category: 'productivity', description: 'Analyze topics and summarize findings.', promptTemplate: 'Research and summarize: {{input}}', priceHint: '$24/mo add-on' },
```

{ slug: 'meeting-summary', name: 'Meeting Summarizer', category: 'productivity', description: 'Extract actions and takeaways.', promptTemplate: 'Summarize this meeting transcript: {{input}}', priceHint: 'Included in Pro' },
{ slug: 'idea-generator', name: 'Idea Generator', category: 'productivity', description: 'Generate startup and campaign ideas.', promptTemplate: 'Generate 25 ideas for: {{input}}', priceHint: '$5/mo add-on' },
{ slug: 'translator', name: 'Translator', category: 'productivity', description: 'Translate with context.', promptTemplate: 'Translate this text preserving tone: {{input}}', priceHint: 'Included in Pro' },
{ slug: 'summarizer', name: 'Summarizer', category: 'productivity', description: 'Compress long text.', promptTemplate: 'Summarize the following text: {{input}}', priceHint: 'Included in Pro' },
{ slug: 'code-generator', name: 'Code Generator', category: 'coding', description: 'Generate code from prompts.', promptTemplate: 'Write production-ready code for: {{input}}', priceHint: '$29/mo add-on' },
{ slug: 'bug-fixer', name: 'Bug Fix Assistant', category: 'coding', description: 'Debug code and explain issues.', promptTemplate: 'Fix and explain this bug: {{input}}', priceHint: '$19/mo add-on' },
{ slug: 'api-docs', name: 'API Docs Generator', category: 'coding', description: 'Generate developer docs.', promptTemplate: 'Write API documentation for: {{input}}', priceHint: '$9/mo add-on' },
{ slug: 'sql-builder', name: 'SQL Query Builder', category: 'coding', description: 'Generate SQL safely.', promptTemplate: 'Write an SQL query for: {{input}}', priceHint: '$12/mo add-on' },
{ slug: 'landing-page-copy', name: 'Landing Page Copy', category: 'coding', description: 'Conversion copy for launches.', promptTemplate: 'Write landing page copy for: {{input}}', priceHint: '$14/mo add-on' },
{ slug: 'website-generator', name: 'Website Generator', category: 'coding', description: 'Generate website structure and sections.', promptTemplate: 'Create website structure and content for: {{input}}', priceHint: '$39/mo add-on' },
{ slug: 'form-builder', name: 'Form Builder Assistant', category: 'coding', description: 'Generate forms and validation logic.', promptTemplate: 'Generate a web form and validation for: {{input}}', priceHint: '$11/mo add-on' },
{ slug: 'workflow-builder', name: 'Workflow Builder', category: 'automation', description: 'Create automation flows.', promptTemplate: 'Design an automation workflow for: {{input}}', priceHint: '$25/mo add-on' },
{ slug: 'email-automation', name: 'Email Automation AI', category: 'automation', description: 'Sequences and triggers.', promptTemplate: 'Create an email automation flow for: {{input}}', priceHint: 'Included in Pro' },
{ slug: 'lead-gen-automation', name: 'Lead Gen Automation', category: 'automation', description: 'Design outbound flows.', promptTemplate: 'Build a lead generation automation for: {{input}}', priceHint: '$29/mo add-on' },
{ slug: 'customer-support-agent', name: 'Customer Support Agent', category: 'automation', description: 'Self-serve support workflows.', promptTemplate: 'Build a support automation for: {{input}}', priceHint: '$35/mo add-on' },

{ slug: 'sales-agent', name: 'Sales Agent', category: 'automation', description: 'Pipeline follow-up automation.', promptTemplate: 'Build a sales automation for: {{input}}', priceHint: '$35/mo add-on' },
{ slug: 'marketing-agent', name: 'Marketing Agent', category: 'automation', description: 'Campaign orchestration.', promptTemplate: 'Build a marketing automation for: {{input}}', priceHint: '$35/mo add-on' },
{ slug: 'social-scheduler', name: 'Social Scheduler', category: 'automation', description: 'Plan and automate posting.', promptTemplate: 'Create a social posting schedule for: {{input}}', priceHint: '$19/mo add-on' },
{ slug: 'seo-cluster-builder', name: 'SEO Cluster Builder', category: 'content', description: 'Topic clusters and internal linking.', promptTemplate: 'Create an SEO content cluster for: {{input}}', priceHint: '$19/mo add-on' },
{ slug: 'competitor-analysis', name: 'Competitor Analysis', category: 'business', description: 'Compare offers and positioning.', promptTemplate: 'Analyze competitors for: {{input}}', priceHint: '$22/mo add-on' },
{ slug: 'pricing-strategy', name: 'Pricing Strategy Generator', category: 'business', description: 'Model offers and plans.', promptTemplate: 'Create a pricing strategy for: {{input}}', priceHint: '$18/mo add-on' },
{ slug: 'onboarding-flow', name: 'Onboarding Flow Designer', category: 'business', description: 'Improve activation rates.', promptTemplate: 'Design an onboarding flow for: {{input}}', priceHint: '$16/mo add-on' },
{ slug: 'survey-analyzer', name: 'Survey Analyzer', category: 'productivity', description: 'Analyze user feedback.', promptTemplate: 'Analyze this survey data: {{input}}', priceHint: '$14/mo add-on' },
{ slug: 'knowledge-base-writer', name: 'Knowledge Base Writer', category: 'business', description: 'Help docs and internal SOPs.', promptTemplate: 'Write a knowledge base article for: {{input}}', priceHint: '$10/mo add-on' },
{ slug: 'headline-generator', name: 'Headline Generator', category: 'content', description: 'Generate headlines and hooks.', promptTemplate: 'Generate 50 headlines for: {{input}}', priceHint: '$7/mo add-on' },
{ slug: 'cold-dm-writer', name: 'Cold DM Writer', category: 'business', description: 'Draft outreach messages.', promptTemplate: 'Write cold DMs for: {{input}}', priceHint: '$9/mo add-on' },
{ slug: 'script-rewriter', name: 'Script Rewriter', category: 'creator', description: 'Improve clarity and pacing.', promptTemplate: 'Rewrite this script for better performance: {{input}}', priceHint: '$13/mo add-on' },
{ slug: 'ugc-brief-generator', name: 'UGC Brief Generator', category: 'creator', description: 'Creator campaign briefs.', promptTemplate: 'Write a UGC creator brief for: {{input}}', priceHint: '$11/mo add-on' },
{ slug: 'micro-saas-idea-generator', name: 'Micro SaaS Idea Generator', category: 'productivity', description: 'Generate startup concepts.', promptTemplate: 'Generate profitable micro-SaaS ideas around: {{input}}', priceHint: '$17/mo add-on' },

```
  { slug: 'agent-marketplace-listing', name: 'Agent Marketplace Listing Writer', category:
'automation', description: 'Write marketplace-ready listings.', promptTemplate: 'Write an agent
marketplace listing for: {{input}}', priceHint: '$8/mo add-on' }
];
```

---

11) components/marketing/ToolMarketplaceGrid.tsx

```
import { aiTools } from '@/data/ai-tools';

export function ToolMarketplaceGrid({ limit }: { limit?: number }) {
  const items = typeof limit === 'number' ? aiTools.slice(0, limit) : aiTools;
  return (
    <section className="container-app py-20">
      <div className="mb-8">
        <h2 className="text-3xl font-bold">50 AI Tools You Can Sell</h2>
        <p className="mt-2 text-zinc-400">A modular marketplace powered by one shared tool
runner API.</p>
      </div>
      <div className="grid gap-4 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4">
        {items.map((tool) => (
          <div key={tool.slug} className="glass rounded-3xl p-5">
            <div className="text-xs uppercase tracking-[0.2em]
text-zinc-500">{tool.category}</div>
            <h3 className="mt-3 text-lg font-semibold">{tool.name}</h3>
            <p className="mt-2 text-sm text-zinc-400">{tool.description}</p>
            <div className="mt-4 text-sm text-zinc-300">{tool.priceHint}</div>
          </div>
        ))}
      </div>
    </section>
  );
}
```

---

12) app/(marketing)/marketplace/page.tsx

```
import { ToolMarketplaceGrid } from '@/components/marketing/ToolMarketplaceGrid';

export default function MarketplacePage() {
  return <ToolMarketplaceGrid />;
}
```

---

13) components/marketing/PricingCards.tsx

```
import Link from 'next/link';
import { plans } from '@/data/plans';

export function PricingCards() {
  return (
    <section className="container-app py-20">
      <div className="mb-8">
        <h2 className="text-3xl font-bold">Simple SaaS Pricing</h2>
        <p className="mt-2 text-zinc-400">Free, Pro, and Enterprise with Stripe checkout.</p>
      </div>
      <div className="grid gap-6 lg:grid-cols-3">
        {plans.map((plan) => (
          <div key={plan.id} className="glass rounded-[2rem] p-6">
            <div className="text-sm uppercase tracking-[0.25em]
text-zinc-500">{plan.name}</div>
            <div className="mt-4 text-4xl font-bold">{plan.price}</div>
            <p className="mt-3 text-zinc-400">{plan.description}</p>
            <ul className="mt-6 space-y-3 text-sm text-zinc-300">
              {plan.features.map((f) => <li key={f}>• {f}</li>)}
            </ul>
            <Link href="/signup" className="mt-6 inline-flex rounded-2xl bg-white px-5 py-3
text-black">
              Choose {plan.name}
            </Link>
          </div>
        ))}
      </div>
    </section>
  );
}
```

---

14) app/(marketing)/pricing/page.tsx

```
import { PricingCards } from '@/components/marketing/PricingCards';

export default function PricingPage() {
  return <PricingCards />;
```

```
}
```

---

15) app/(auth)/login/page.tsx

```
export default function LoginPage() {
  return (
    <main className="container-app py-20">
      <div className="mx-auto max-w-md rounded-[2rem] border border-white/10 bg-white/5
p-8">
        <h1 className="text-3xl font-bold">Login</h1>
        <form className="mt-6 space-y-4">
          <input className="w-full rounded-2xl border border-white/10 bg-zinc-900 px-4 py-3"
placeholder="Email" />
          <input className="w-full rounded-2xl border border-white/10 bg-zinc-900 px-4 py-3"
placeholder="Password" type="password" />
          <button className="w-full rounded-2xl bg-white px-4 py-3 text-black">Sign In</button>
        </form>
      </div>
    </main>
  );
}
```

---

16) app/(auth)/signup/page.tsx

```
export default function SignupPage() {
  return (
    <main className="container-app py-20">
      <div className="mx-auto max-w-md rounded-[2rem] border border-white/10 bg-white/5
p-8">
        <h1 className="text-3xl font-bold">Create Account</h1>
        <form className="mt-6 space-y-4">
          <input className="w-full rounded-2xl border border-white/10 bg-zinc-900 px-4 py-3"
placeholder="Full name" />
          <input className="w-full rounded-2xl border border-white/10 bg-zinc-900 px-4 py-3"
placeholder="Email" />
          <input className="w-full rounded-2xl border border-white/10 bg-zinc-900 px-4 py-3"
placeholder="Password" type="password" />
          <button className="w-full rounded-2xl bg-white px-4 py-3 text-black">Start
Free</button>
        </form>
```

```
      </div>
    </main>
  );
}
```

---

17) components/dashboard/Sidebar.tsx

```
import Link from 'next/link';

const items = [
  ['Overview', '/dashboard'],
  ['Tools', '/dashboard/tools'],
  ['Agents', '/dashboard/agents'],
  ['Automation', '/dashboard/automation'],
  ['Verification', '/dashboard/verification'],
  ['Billing', '/dashboard/billing'],
  ['API Keys', '/dashboard/api-keys'],
  ['Settings', '/dashboard/settings']
];

export function Sidebar() {
  return (
    <aside className="hidden w-72 shrink-0 border-r border-white/10 p-6 lg:block">
      <div className="mb-8 text-xl font-bold">Dashboard</div>
      <nav className="space-y-2">
        {items.map(([label, href]) => (
          <Link key={href} href={href} className="block rounded-2xl px-4 py-3 text-zinc-300
hover:bg-white/5">
            {label}
          </Link>
        ))}
      </nav>
    </aside>
  );
}
```

---

18) components/dashboard/DashboardShell.tsx

```
import { Sidebar } from './Sidebar';
```

```
export function DashboardShell({ title, children }: { title: string; children: React.ReactNode }) {
  return (
    <div className="min-h-screen lg:flex">
      <Sidebar />
      <main className="flex-1 p-6 lg:p-10">
        <div className="mb-8 flex items-center justify-between">
          <h1 className="text-3xl font-bold">{title}</h1>
          <button className="rounded-2xl bg-white px-4 py-2 text-black">Upgrade</button>
        </div>
        {children}
      </main>
    </div>
  );
}
```

---

19) app/dashboard/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

export default function DashboardPage() {
  return (
    <DashboardShell title="Overview">
      <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-4">
        {[
          ['Total Tool Runs', '12,480'],
          ['Agents Active', '8'],
          ['Verification Jobs', '126'],
          ['MRR Potential', '$18,240']
        ].map(([label, value]) => (
          <div key={label} className="glass rounded-3xl p-6">
            <div className="text-sm text-zinc-400">{label}</div>
            <div className="mt-2 text-3xl font-bold">{value}</div>
          </div>
        ))}
      </div>
    </DashboardShell>
  );
}
```

---

20) components/dashboard/ToolRunner.tsx

```
'use client';

import { useState } from 'react';
import { aiTools } from '@/data/ai-tools';

export function ToolRunner() {
  const [toolSlug, setToolSlug] = useState(aiTools[0].slug);
  const [input, setInput] = useState('');
  const [output, setOutput] = useState('');
  const [loading, setLoading] = useState(false);

  async function runTool() {
    setLoading(true);
    setOutput('');
    const res = await fetch('/api/tools/run', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ toolSlug, input })
    });
    const data = await res.json();
    setOutput(data.output || 'No output');
    setLoading(false);
  }

  return (
    <div className="grid gap-6 lg:grid-cols-[320px,1fr]">
      <div className="glass rounded-3xl p-5">
        <label className="mb-2 block text-sm text-zinc-400">Choose Tool</label>
        <select className="w-full rounded-2xl bg-zinc-900 p-3" value={toolSlug} onChange={(e)
=> setToolSlug(e.target.value)}>
          {aiTools.map((tool) => <option key={tool.slug} value={tool.slug}>{tool.name}</option>)}
        </select>
        <textarea
          className="mt-4 min-h-[220px] w-full rounded-2xl bg-zinc-900 p-3"
          placeholder="Describe what you want the AI to do"
          value={input}
          onChange={(e) => setInput(e.target.value)}
        />
        <button onClick={runTool} className="mt-4 w-full rounded-2xl bg-white px-4 py-3
text-black">
          {loading ? 'Running...' : 'Run Tool'}
        </button>
```

```
      </div>
      <div className="glass rounded-3xl p-5">
        <div className="mb-3 text-sm text-zinc-400">Output</div>
        <pre className="whitespace-pre-wrap text-sm text-zinc-200">{output}</pre>
      </div>
    </div>
  );
}
```

---

21) app/dashboard/tools/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';
import { ToolRunner } from '@/components/dashboard/ToolRunner';

export default function DashboardToolsPage() {
  return (
    <DashboardShell title="AI Tools">
      <ToolRunner />
    </DashboardShell>
  );
}
```

---

22) data/agent-templates.ts

```
export const agentTemplates = [
  { slug: 'support-agent', name: 'Customer Support Agent', role: 'Resolve FAQs, triage tickets, and draft replies.' },
  { slug: 'marketing-agent', name: 'Marketing Agent', role: 'Generate campaigns, hooks, and promotional calendars.' },
  { slug: 'sales-agent', name: 'Sales Agent', role: 'Follow up with leads, score deals, and create scripts.' },
  { slug: 'lead-agent', name: 'Lead Generation Agent', role: 'Identify prospects, qualify leads, and suggest outreach.' },
  { slug: 'email-agent', name: 'Email Automation Agent', role: 'Build sequences and run drip campaigns.' },
  { slug: 'social-agent', name: 'Social Media Agent', role: 'Schedule and generate social content at scale.' },
  { slug: 'site-agent', name: 'Website Builder Agent', role: 'Generate sitemap, copy, and CTA strategy.' },
```

```
  { slug: 'research-agent', name: 'Research Agent', role: 'Analyze industries, trends, and positioning.' },
  { slug: 'ops-agent', name: 'Operations Agent', role: 'Generate SOPs, forms, and internal workflows.' },
  { slug: 'growth-agent', name: 'Growth Hacking Agent', role: 'Test offers, channels, and experiments.' }
];
```

---

23) components/dashboard/AgentBuilder.tsx

```
'use client';

import { useState } from 'react';
import { agentTemplates } from '@/data/agent-templates';

export function AgentBuilder() {
  const [name, setName] = useState('My Revenue Agent');
  const [template, setTemplate] = useState(agentTemplates[0].slug);
  const [goal, setGoal] = useState('Help convert more leads into sales.');
  const [result, setResult] = useState('');

  async function createAgent() {
    const res = await fetch('/api/agents/create', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name, template, goal })
    });
    const data = await res.json();
    setResult(JSON.stringify(data, null, 2));
  }

  return (
    <div className="grid gap-6 lg:grid-cols-[360px,1fr]">
      <div className="glass rounded-3xl p-5">
        <label className="mb-2 block text-sm text-zinc-400">Agent Name</label>
        <input className="w-full rounded-2xl bg-zinc-900 p-3" value={name} onChange={(e) =>
setName(e.target.value)} />
        <label className="mb-2 mt-4 block text-sm text-zinc-400">Template</label>
        <select className="w-full rounded-2xl bg-zinc-900 p-3" value={template} onChange={(e)
=> setTemplate(e.target.value)}>
          {agentTemplates.map((t) => <option key={t.slug} value={t.slug}>{t.name}</option>)}
        </select>
```

```
        <label className="mb-2 mt-4 block text-sm text-zinc-400">Goal</label>
        <textarea className="min-h-[180px] w-full rounded-2xl bg-zinc-900 p-3" value={goal}
onChange={(e) => setGoal(e.target.value)} />
        <button onClick={createAgent} className="mt-4 w-full rounded-2xl bg-white px-4 py-3
text-black">Create Agent</button>
      </div>
      <div className="glass rounded-3xl p-5">
        <div className="mb-3 text-sm text-zinc-400">Agent JSON Output</div>
        <pre className="whitespace-pre-wrap text-sm text-zinc-200">{result}</pre>
      </div>
    </div>
  );
}
```

---

24) app/dashboard/agents/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';
import { AgentBuilder } from '@/components/dashboard/AgentBuilder';

export default function DashboardAgentsPage() {
  return (
    <DashboardShell title="AI Agent Builder">
      <AgentBuilder />
    </DashboardShell>
  );
}
```

---

25) components/dashboard/VerificationJobForm.tsx

```
'use client';

import { useState } from 'react';

export function VerificationJobForm() {
  const [category, setCategory] = useState('fact-checking');
  const [content, setContent] = useState('');
  const [response, setResponse] = useState('');

  async function submitJob() {
    const res = await fetch('/api/verification/jobs', {
```

```
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ category, content })
    });
    const data = await res.json();
    setResponse(`Job created: ${data.jobId}`);
  }

  return (
    <div className="glass rounded-3xl p-5">
      <h2 className="text-xl font-semibold">Submit Verification Job</h2>
      <select className="mt-4 w-full rounded-2xl bg-zinc-900 p-3" value={category}
onChange={(e) => setCategory(e.target.value)}>
        {['content-moderation', 'fact-checking', 'spam-detection', 'quality-control', 'image-review',
'video-moderation', 'ai-hallucination-detection'].map((c) => (
          <option key={c} value={c}>{c}</option>
        ))}
      </select>
      <textarea
        className="mt-4 min-h-[220px] w-full rounded-2xl bg-zinc-900 p-3"
        value={content}
        onChange={(e) => setContent(e.target.value)}
        placeholder="Paste the AI output or content to verify"
      />
      <button onClick={submitJob} className="mt-4 rounded-2xl bg-white px-4 py-3
text-black">Submit for Review</button>
      <div className="mt-4 text-sm text-zinc-400">{response}</div>
    </div>
  );
}
```

---

26) components/dashboard/VerificationMarketplace.tsx

```
export function VerificationMarketplace() {
  const workers = [
    { name: 'Moderator One', rating: 4.9, jobs: 1200, specialty: 'Content Moderation' },
    { name: 'Checker Pro', rating: 4.8, jobs: 860, specialty: 'Fact Checking' },
    { name: 'Vision Reviewer', rating: 4.7, jobs: 510, specialty: 'Image Review' }
  ];

  return (
    <div className="glass rounded-3xl p-5">
```

```
      <h2 className="text-xl font-semibold">Human Verification Marketplace</h2>
      <div className="mt-5 space-y-4">
        {workers.map((worker) => (
          <div key={worker.name} className="rounded-2xl border border-white/10 p-4">
            <div className="flex flex-wrap items-center justify-between gap-3">
              <div>
                <div className="font-semibold">{worker.name}</div>
                <div className="text-sm text-zinc-400">{worker.specialty}</div>
              </div>
              <div className="text-sm text-zinc-300">⭐ {worker.rating} • {worker.jobs} jobs</div>
            </div>
          </div>
        ))}
      </div>
    </div>
  );
}
```

---

27) app/dashboard/verification/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';
import { VerificationJobForm } from '@/components/dashboard/VerificationJobForm';
import { VerificationMarketplace } from '@/components/dashboard/VerificationMarketplace';

export default function DashboardVerificationPage() {
  return (
    <DashboardShell title="Human Verification">
      <div className="grid gap-6 lg:grid-cols-2">
        <VerificationJobForm />
        <VerificationMarketplace />
      </div>
    </DashboardShell>
  );
}
```

---

28) app/(marketing)/human-verification/page.tsx

```
export default function HumanVerificationPage() {
  return (
    <main className="container-app py-20">
```

```
      <div className="max-w-4xl">
        <h1 className="text-5xl font-bold">Human Verification Marketplace</h1>
        <p className="mt-6 text-lg text-zinc-400">
          Add a human-in-the-loop layer to fact-check, moderate, score quality, and review AI
outputs before they reach customers.
        </p>
      </div>
      <div className="mt-10 grid gap-4 md:grid-cols-2 xl:grid-cols-4">
        {['Fact Checking', 'Spam Detection', 'Quality Control', 'Image Review'].map((item) => (
          <div key={item} className="glass rounded-3xl p-5">
            <div className="text-xl font-semibold">{item}</div>
            <p className="mt-2 text-sm text-zinc-400">Marketplace-ready workflow with jobs,
worker ratings, and review queues.</p>
          </div>
        ))}
      </div>
    </main>
  );
}
```

---

29) app/(marketing)/automation-agents/page.tsx

```
import { agentTemplates } from '@/data/agent-templates';

export default function AutomationAgentsPage() {
  return (
    <main className="container-app py-20">
      <h1 className="text-5xl font-bold">Automation Agents</h1>
      <p className="mt-4 max-w-2xl text-lg text-zinc-400">
        Deploy agents for sales, support, growth, operations, and content workflows.
      </p>
      <div className="mt-10 grid gap-4 md:grid-cols-2 xl:grid-cols-3">
        {agentTemplates.map((agent) => (
          <div key={agent.slug} className="glass rounded-3xl p-5">
            <h2 className="text-xl font-semibold">{agent.name}</h2>
            <p className="mt-2 text-sm text-zinc-400">{agent.role}</p>
          </div>
        ))}
      </div>
    </main>
  );
}
```

---

30) lib/openai.ts

```
import OpenAI from 'openai';
import { env } from './env';

export const openai = new OpenAI({
  apiKey: env.OPENAI_API_KEY
});
```

---

31) lib/env.ts

```
function getEnv(name: string, fallback?: string) {
  const value = process.env[name] || fallback;
  if (!value) throw new Error(`Missing environment variable: ${name}`);
  return value;
}

export const env = {
  NEXT_PUBLIC_APP_URL: getEnv('NEXT_PUBLIC_APP_URL'),
  NEXT_PUBLIC_SUPABASE_URL: getEnv('NEXT_PUBLIC_SUPABASE_URL'),
  NEXT_PUBLIC_SUPABASE_ANON_KEY:
getEnv('NEXT_PUBLIC_SUPABASE_ANON_KEY'),
  SUPABASE_SERVICE_ROLE_KEY: getEnv('SUPABASE_SERVICE_ROLE_KEY'),
  OPENAI_API_KEY: getEnv('OPENAI_API_KEY'),
  STRIPE_SECRET_KEY: getEnv('STRIPE_SECRET_KEY'),
  STRIPE_WEBHOOK_SECRET: getEnv('STRIPE_WEBHOOK_SECRET')
};
```

---

32) lib/supabase-server.ts

```
import { createClient } from '@supabase/supabase-js';
import { env } from './env';

export function createAdminSupabaseClient() {
  return createClient(env.NEXT_PUBLIC_SUPABASE_URL,
env.SUPABASE_SERVICE_ROLE_KEY);
}
```

---

33) lib/supabase-client.ts

```
import { createClient } from '@supabase/supabase-js';
import { env } from './env';

export const supabaseBrowser = createClient(
  env.NEXT_PUBLIC_SUPABASE_URL,
  env.NEXT_PUBLIC_SUPABASE_ANON_KEY
);
```

---

34) lib/stripe.ts

```
import Stripe from 'stripe';
import { env } from './env';

export const stripe = new Stripe(env.STRIPE_SECRET_KEY, {
  apiVersion: '2025-02-24.acacia'
});
```

---

35) app/api/tools/run/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { aiTools } from '@/data/ai-tools';
import { openai } from '@/lib/openai';

export async function POST(request: NextRequest) {
  try {
    const { toolSlug, input } = await request.json();
    const tool = aiTools.find((t) => t.slug === toolSlug);

    if (!tool) {
      return NextResponse.json({ error: 'Tool not found' }, { status: 404 });
    }

    const prompt = tool.promptTemplate.replace('{{input}}', input || '');

    const completion = await openai.responses.create({
```

```
      model: 'gpt-4.1-mini',
      input: prompt
    });

    const output = completion.output_text || 'No output generated.';

    return NextResponse.json({ success: true, tool: tool.name, output });
  } catch (error) {
    return NextResponse.json({ error: 'Failed to run tool' }, { status: 500 });
  }
}
```

---

36) app/api/agents/create/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { agentTemplates } from '@/data/agent-templates';

export async function POST(request: NextRequest) {
  const body = await request.json();
  const template = agentTemplates.find((t) => t.slug === body.template);

  return NextResponse.json({
    success: true,
    agent: {
      id: crypto.randomUUID(),
      name: body.name,
      template: body.template,
      goal: body.goal,
      config: {
        role: template?.role,
        triggers: ['manual', 'schedule', 'webhook'],
        actions: ['generate', 'analyze', 'reply', 'summarize']
      }
    }
  });
}
```

---

37) app/api/agents/run/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
```

```
import { openai } from '@/lib/openai';

export async function POST(request: NextRequest) {
  const { agentName, goal, context } = await request.json();

  const prompt = `You are ${agentName}. Your goal is: ${goal}. Use this context: ${context}`;

  const completion = await openai.responses.create({
    model: 'gpt-4.1-mini',
    input: prompt
  });

  return NextResponse.json({
    success: true,
    output: completion.output_text
  });
}
```

---

38) app/api/verification/jobs/route.ts

```
import { NextRequest, NextResponse } from 'next/server';

export async function POST(request: NextRequest) {
  const { category, content } = await request.json();

  return NextResponse.json({
    success: true,
    jobId: crypto.randomUUID(),
    category,
    status: 'queued',
    preview: String(content || '').slice(0, 160)
  });
}
```

---

39) app/api/verification/worker-apply/route.ts

```
import { NextRequest, NextResponse } from 'next/server';

export async function POST(request: NextRequest) {
  const { name, email, specialty } = await request.json();
```

```
  return NextResponse.json({
    success: true,
    applicationId: crypto.randomUUID(),
    name,
    email,
    specialty,
    status: 'pending_review'
  });
}
```

---

40) app/api/billing/checkout/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { stripe } from '@/lib/stripe';

export async function POST(request: NextRequest) {
  const { priceId } = await request.json();

  const session = await stripe.checkout.sessions.create({
    mode: 'subscription',
    line_items: [{ price: priceId, quantity: 1 }],
    success_url: `${process.env.NEXT_PUBLIC_APP_URL}/dashboard/billing?success=true`,
    cancel_url: `${process.env.NEXT_PUBLIC_APP_URL}/pricing?canceled=true`
  });

  return NextResponse.json({ url: session.url });
}
```

---

41) app/api/billing/portal/route.ts

```
import { NextResponse } from 'next/server';

export async function POST() {
  return NextResponse.json({
    success: true,
    message: 'Attach Stripe customer portal here once customer IDs are persisted.'
  });
}
```

---

42) app/api/stripe/webhook/route.ts

```
import { NextRequest, NextResponse } from 'next/server';

export async function POST(_request: NextRequest) {
  return NextResponse.json({ received: true });
}
```

---

43) app/dashboard/billing/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';
import { plans } from '@/data/plans';

export default function BillingPage() {
  return (
    <DashboardShell title="Billing">
      <div className="grid gap-6 lg:grid-cols-3">
        {plans.map((plan) => (
          <div key={plan.id} className="glass rounded-3xl p-6">
            <div className="text-xl font-semibold">{plan.name}</div>
            <div className="mt-2 text-4xl font-bold">{plan.price}</div>
            <ul className="mt-4 space-y-2 text-sm text-zinc-300">
              {plan.features.map((feature) => <li key={feature}>• {feature}</li>)}
            </ul>
            <button className="mt-6 rounded-2xl bg-white px-4 py-3
text-black">Checkout</button>
          </div>
        ))}
      </div>
    </DashboardShell>
  );
}
```

---

44) app/dashboard/api-keys/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

export default function ApiKeysPage() {
```

```
  return (
    <DashboardShell title="API Keys">
      <div className="glass rounded-3xl p-6">
        <h2 className="text-xl font-semibold">Developer Access</h2>
        <p className="mt-2 text-zinc-400">Generate API keys so customers can embed
Superagent Jroc AI into other apps.</p>
        <div className="mt-6 flex gap-4">
          <button className="rounded-2xl bg-white px-4 py-3 text-black">Generate Key</button>
          <div className="rounded-2xl border border-white/10 px-4 py-3 text-sm
text-zinc-300">sk_live_jroc_xxxxxxxxx</div>
        </div>
      </div>
    </DashboardShell>
  );
}
```

---

45) app/dashboard/settings/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

export default function SettingsPage() {
  return (
    <DashboardShell title="Settings">
      <div className="glass rounded-3xl p-6">
        <h2 className="text-xl font-semibold">Account Settings</h2>
        <div className="mt-6 grid gap-4 md:grid-cols-2">
          <input className="rounded-2xl bg-zinc-900 p-3" placeholder="Business Name" />
          <input className="rounded-2xl bg-zinc-900 p-3" placeholder="Website" />
          <input className="rounded-2xl bg-zinc-900 p-3" placeholder="Support Email" />
          <input className="rounded-2xl bg-zinc-900 p-3" placeholder="Timezone" />
        </div>
        <button className="mt-6 rounded-2xl bg-white px-4 py-3 text-black">Save
Changes</button>
      </div>
    </DashboardShell>
  );
}
```

---

46) app/admin/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

export default function AdminPage() {
  return (
    <DashboardShell title="Admin Panel">
      <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-4">
        {[
          ['Users', '4,208'],
          ['MRR', '$32,940'],
          ['Workers', '314'],
          ['Jobs Today', '1,482']
        ].map(([label, value]) => (
          <div key={label} className="glass rounded-3xl p-6">
            <div className="text-sm text-zinc-400">{label}</div>
            <div className="mt-2 text-3xl font-bold">{value}</div>
          </div>
        ))}
      </div>
    </DashboardShell>
  );
}
```

---

47) db/schema.sql

```
create extension if not exists pgcrypto;

create table if not exists profiles (
  id uuid primary key default gen_random_uuid(),
  email text unique not null,
  full_name text,
  role text default 'user',
  created_at timestamptz default now()
);

create table if not exists subscriptions (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id) on delete cascade,
  stripe_customer_id text,
  stripe_subscription_id text,
  plan text not null default 'free',
  status text default 'inactive',
  created_at timestamptz default now()
```

```
);

create table if not exists api_keys (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id) on delete cascade,
  key_hash text not null,
  label text,
  last_used_at timestamptz,
  created_at timestamptz default now()
);

create table if not exists usage_logs (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id) on delete cascade,
  tool_slug text,
  run_type text,
  units integer default 1,
  metadata jsonb default '{}'::jsonb,
  created_at timestamptz default now()
);

create table if not exists agents (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id) on delete cascade,
  name text not null,
  template_slug text,
  goal text,
  config jsonb default '{}'::jsonb,
  status text default 'draft',
  created_at timestamptz default now()
);

create table if not exists automation_runs (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id) on delete cascade,
  agent_id uuid references agents(id) on delete cascade,
  status text default 'queued',
  input jsonb default '{}'::jsonb,
  output jsonb default '{}'::jsonb,
  created_at timestamptz default now()
);

create table if not exists verification_workers (
  id uuid primary key default gen_random_uuid(),
```

```
  user_id uuid references profiles(id) on delete cascade,
  specialty text,
  rating numeric(3,2) default 5.0,
  total_jobs integer default 0,
  status text default 'pending',
  created_at timestamptz default now()
);

create table if not exists verification_jobs (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references profiles(id) on delete cascade,
  category text not null,
  content text not null,
  status text default 'queued',
  assigned_worker_id uuid references verification_workers(id),
  result jsonb default '{}'::jsonb,
  created_at timestamptz default now()
);

create table if not exists payouts (
  id uuid primary key default gen_random_uuid(),
  worker_id uuid references verification_workers(id) on delete cascade,
  amount_cents integer not null,
  status text default 'pending',
  created_at timestamptz default now()
);
```

---

48) README.md

```
# Superagent Jroc AI

A modern AI SaaS platform with:
- 50 AI tools marketplace
- Agent builder
- Human verification marketplace
- Stripe subscriptions
- API keys
- Admin dashboard

## Local Setup

1. Clone repo
```

2. Copy `.env.example` to `.env.local`
3. Add Supabase, Stripe, and OpenAI keys
4. Install packages:
   ```bash
   npm install

Run dev:
 npm run dev

   5.

# Deploy

- Frontend: Vercel
- Database/Auth: Supabase
- Billing: Stripe

# Monetization

- SaaS subscriptions
- Tool add-ons
- Agent marketplace commissions
- Verification job fees
- API usage revenue

---

## 49) $10M Startup Architecture

### Core Revenue Engines
1. **Subscriptions** — Free / Pro / Enterprise
2. **Add-on tools** — Sell premium tools by category
3. **Verification fees** — Charge per human-reviewed job
4. **Agent marketplace** — Take commission on community agents
5. **API access** — Usage-based pricing for developers
6. **Enterprise onboarding** — White-label and setup fees

### Growth Flywheel
- Users join for one tool
- Upgrade for workflows
- Add agents to save labor
- Add human review for trust
   ```

- Invite workers and creators
- Launch agent marketplace
- Sell API to businesses

Moat
- AI + human verification
- Multi-tool bundle under one login
- Marketplace network effects
- Usage data for improving templates
- Enterprise workflow lock-in

12-Month Expansion
- Team accounts
- Affiliate/referral system
- White-label dashboards
- Agent marketplace with reviews
- Vertical packs: real estate, ecommerce, agencies, creators

$10M Path Example
- 3,000 Pro users at $29/mo = $87,000 MRR
- 500 Enterprise users at $99/mo = $49,500 MRR
- Add-ons + API + marketplace + verification = $100k–$250k MRR upside

---

50) What To Build Next

Highest-impact next files
1. Supabase auth flow with protected routes
2. Stripe webhook persistence
3. Real user billing state in dashboard
4. Agent run logs and scheduler
5. Worker payout logic
6. Admin moderation queue
7. Affiliate/referral system
8. White-label settings

Best scaling decision
Use **one universal tool runner** with the `aiTools` config file rather than hand-coding 50 separate backends. That gives you 50 sellable modules with much less maintenance.

---

51) Copy-Paste Prompt for GitHub Copilot

```text
Build the Superagent Jroc AI repo exactly as specified in this document.
Use Next.js 14 App Router, TypeScript, Tailwind, Supabase, Stripe, and OpenAI.
Keep the dark premium dashboard styling.
Implement all routes and components listed.
Use the aiTools config file as the source of truth for the 50 tool modules.
Use a single /api/tools/run endpoint that loads prompt templates dynamically by slug.
Create production-safe code with reusable components, clean types, and organized folders.
Add protected dashboard routes, a working Stripe checkout flow, and Supabase-ready schema integration.
```

---

52) Phase 2 — Real Auth, Route Protection, Billing Wiring, and Deploy

Below are the next files to make the starter repo feel much more real.

middleware.ts

```
import { NextResponse, type NextRequest } from 'next/server';

const protectedPaths = ['/dashboard', '/admin'];

export function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;
  const needsAuth = protectedPaths.some((path) => pathname.startsWith(path));

  const session = request.cookies.get('sjroc_session')?.value;

  if (needsAuth && !session) {
    const url = request.nextUrl.clone();
    url.pathname = '/login';
    return NextResponse.redirect(url);
  }

  return NextResponse.next();
}

export const config = {
  matcher: ['/dashboard/:path*', '/admin/:path*']
};
```

lib/auth.ts

```
import { cookies } from 'next/headers';

export async function getSessionUser() {
  const cookieStore = await cookies();
  const raw = cookieStore.get('sjroc_session')?.value;
  if (!raw) return null;

  try {
    return JSON.parse(raw);
  } catch {
    return null;
  }
}

export async function requireUser() {
  const user = await getSessionUser();
  if (!user) {
    throw new Error('Unauthorized');
  }
  return user;
}
```

app/api/auth/session/route.ts

```
import { NextRequest, NextResponse } from 'next/server';

export async function POST(request: NextRequest) {
  const { email, password } = await request.json();

  if (!email || !password) {
    return NextResponse.json({ error: 'Missing credentials' }, { status: 400 });
  }

  const response = NextResponse.json({
    success: true,
    user: {
      id: crypto.randomUUID(),
      email,
      role: email.includes('admin') ? 'admin' : 'user'
    }
  });
```

```
  response.cookies.set(
    'sjroc_session',
    JSON.stringify({
      id: crypto.randomUUID(),
      email,
      role: email.includes('admin') ? 'admin' : 'user'
    }),
    {
      httpOnly: true,
      sameSite: 'lax',
      secure: process.env.NODE_ENV === 'production',
      path: '/'
    }
  );

  return response;
}

export async function DELETE() {
  const response = NextResponse.json({ success: true });
  response.cookies.set('sjroc_session', '', {
    httpOnly: true,
    expires: new Date(0),
    path: '/'
  });
  return response;
}
```

app/(auth)/login/page.tsx

```
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';

export default function LoginPage() {
  const router = useRouter();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  async function handleLogin(e: React.FormEvent) {
    e.preventDefault();
```

```
    setLoading(true);
    setError('');

    const res = await fetch('/api/auth/session', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password })
    });

    const data = await res.json();

    if (!res.ok) {
      setError(data.error || 'Login failed');
      setLoading(false);
      return;
    }

    router.push('/dashboard');
    router.refresh();
  }

  return (
    <main className="container-app py-20">
      <div className="mx-auto max-w-md rounded-[2rem] border border-white/10 bg-white/5
p-8">
        <h1 className="text-3xl font-bold">Login</h1>
        <form className="mt-6 space-y-4" onSubmit={handleLogin}>
          <input
            className="w-full rounded-2xl border border-white/10 bg-zinc-900 px-4 py-3"
            placeholder="Email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
          />
          <input
            className="w-full rounded-2xl border border-white/10 bg-zinc-900 px-4 py-3"
            placeholder="Password"
            type="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
          />
          {error ? <div className="text-sm text-red-400">{error}</div> : null}
          <button className="w-full rounded-2xl bg-white px-4 py-3 text-black">
            {loading ? 'Signing in...' : 'Sign In'}
          </button>
```

```
        </form>
      </div>
    </main>
  );
}
```

app/api/health/route.ts

```
import { NextResponse } from 'next/server';

export async function GET() {
  return NextResponse.json({
    ok: true,
    service: 'Superagent Jroc AI',
    timestamp: new Date().toISOString()
  });
}
```

lib/pricing.ts

```
export const STRIPE_PRICE_MAP = {
  free: process.env.STRIPE_PRICE_FREE || 'price_free',
  pro: process.env.STRIPE_PRICE_PRO || 'price_pro',
  enterprise: process.env.STRIPE_PRICE_ENTERPRISE || 'price_enterprise'
};
```

app/dashboard/billing/page.tsx

```
'use client';

import { DashboardShell } from '@/components/dashboard/DashboardShell';
import { plans } from '@/data/plans';
import { STRIPE_PRICE_MAP } from '@/lib/pricing';

export default function BillingPage() {
  async function startCheckout(planId: 'free' | 'pro' | 'enterprise') {
    if (planId === 'free') return;

    const res = await fetch('/api/billing/checkout', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ priceId: STRIPE_PRICE_MAP[planId] })
    });
```

```
    const data = await res.json();
    if (data.url) {
      window.location.href = data.url;
    }
  }

  return (
    <DashboardShell title="Billing">
      <div className="grid gap-6 lg:grid-cols-3">
        {plans.map((plan) => (
          <div key={plan.id} className="glass rounded-3xl p-6">
            <div className="text-xl font-semibold">{plan.name}</div>
            <div className="mt-2 text-4xl font-bold">{plan.price}</div>
            <ul className="mt-4 space-y-2 text-sm text-zinc-300">
              {plan.features.map((feature) => <li key={feature}>• {feature}</li>)}
            </ul>
            <button
              onClick={() => startCheckout(plan.id as 'free' | 'pro' | 'enterprise')}
              className="mt-6 rounded-2xl bg-white px-4 py-3 text-black"
            >
              {plan.id === 'free' ? 'Current Plan' : `Upgrade to ${plan.name}`}
            </button>
          </div>
        ))}
      </div>
    </DashboardShell>
  );
}
```

app/api/stripe/webhook/route.ts

```
import { headers } from 'next/headers';
import { NextRequest, NextResponse } from 'next/server';
import { stripe } from '@/lib/stripe';

export async function POST(request: NextRequest) {
  const body = await request.text();
  const signature = (await headers()).get('stripe-signature');

  if (!signature) {
    return NextResponse.json({ error: 'Missing stripe signature' }, { status: 400 });
  }
```

```
  try {
    const event = stripe.webhooks.constructEvent(
      body,
      signature,
      process.env.STRIPE_WEBHOOK_SECRET as string
    );

    switch (event.type) {
      case 'checkout.session.completed':
        break;
      case 'customer.subscription.updated':
        break;
      case 'customer.subscription.deleted':
        break;
      default:
        break;
    }

    return NextResponse.json({ received: true, type: event.type });
  } catch (error) {
    return NextResponse.json({ error: 'Invalid webhook' }, { status: 400 });
  }
}
```

app/dashboard/automation/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

const flows = [
  {
    name: 'Lead Follow-up Flow',
    trigger: 'New inbound lead',
    action: 'Score lead, write intro email, queue human review'
  },
  {
    name: 'Content Approval Flow',
    trigger: 'Blog post generated',
    action: 'Run SEO tool, summarize changes, submit verification job'
  },
  {
    name: 'Client Onboarding Flow',
    trigger: 'Stripe checkout completed',
    action: 'Create workspace, provision API key, start onboarding emails'
  }
```

```
];

export default function AutomationPage() {
  return (
    <DashboardShell title="Automation Workflows">
      <div className="grid gap-4 xl:grid-cols-3">
        {flows.map((flow) => (
          <div key={flow.name} className="glass rounded-3xl p-6">
            <div className="text-xl font-semibold">{flow.name}</div>
            <div className="mt-4 text-sm text-zinc-400">Trigger</div>
            <div className="mt-1 text-zinc-200">{flow.trigger}</div>
            <div className="mt-4 text-sm text-zinc-400">Actions</div>
            <div className="mt-1 text-zinc-200">{flow.action}</div>
          </div>
        ))}
      </div>
    </DashboardShell>
  );
}
```

app/admin/users/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

const users = [
  { email: 'owner@superagentjroc.ai', plan: 'Enterprise', status: 'active' },
  { email: 'growth@client.com', plan: 'Pro', status: 'active' },
  { email: 'creator@brand.com', plan: 'Free', status: 'trial' }
];

export default function AdminUsersPage() {
  return (
    <DashboardShell title="Admin Users">
      <div className="glass rounded-3xl overflow-hidden">
        <table className="w-full text-left text-sm">
          <thead className="border-b border-white/10 text-zinc-400">
            <tr>
              <th className="p-4">Email</th>
              <th className="p-4">Plan</th>
              <th className="p-4">Status</th>
            </tr>
          </thead>
          <tbody>
            {users.map((user) => (
```

```
            <tr key={user.email} className="border-b border-white/5">
              <td className="p-4">{user.email}</td>
              <td className="p-4">{user.plan}</td>
              <td className="p-4">{user.status}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </DashboardShell>
 );
}
```

app/admin/payments/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

const payments = [
 { id: 'pay_001', email: 'owner@superagentjroc.ai', amount: '$99.00', status: 'paid' },
 { id: 'pay_002', email: 'growth@client.com', amount: '$29.00', status: 'paid' },
 { id: 'pay_003', email: 'trial@brand.com', amount: '$0.00', status: 'free' }
];

export default function AdminPaymentsPage() {
 return (
  <DashboardShell title="Admin Payments">
    <div className="grid gap-4">
      {payments.map((payment) => (
        <div key={payment.id} className="glass rounded-3xl p-5 flex items-center
justify-between gap-4">
          <div>
            <div className="font-semibold">{payment.email}</div>
            <div className="text-sm text-zinc-400">{payment.id}</div>
          </div>
          <div className="text-zinc-200">{payment.amount}</div>
          <div className="text-sm text-zinc-400">{payment.status}</div>
        </div>
      ))}
    </div>
  </DashboardShell>
 );
}
```

app/admin/workers/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

const workers = [
  { name: 'Moderator One', specialty: 'Content Moderation', rating: 4.9, status: 'approved' },
  { name: 'Checker Pro', specialty: 'Fact Checking', rating: 4.8, status: 'approved' },
  { name: 'New Reviewer', specialty: 'Image Review', rating: 5.0, status: 'pending' }
];

export default function AdminWorkersPage() {
  return (
    <DashboardShell title="Admin Workers">
      <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-3">
        {workers.map((worker) => (
          <div key={worker.name} className="glass rounded-3xl p-6">
            <div className="text-xl font-semibold">{worker.name}</div>
            <div className="mt-2 text-sm text-zinc-400">{worker.specialty}</div>
            <div className="mt-4 text-zinc-200">Rating: {worker.rating}</div>
            <div className="mt-2 text-zinc-400">Status: {worker.status}</div>
          </div>
        ))}
      </div>
    </DashboardShell>
  );
}
```

app/admin/analytics/page.tsx

```
import { DashboardShell } from '@/components/dashboard/DashboardShell';

const metrics = [
  ['MRR', '$132,400'],
  ['Tool Runs', '280,442'],
  ['Verification Jobs', '14,201'],
  ['Avg Revenue per User', '$41.70'],
  ['Churn', '3.1%'],
  ['Marketplace GMV', '$58,340']
];

export default function AdminAnalyticsPage() {
  return (
    <DashboardShell title="Admin Analytics">
      <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-3">
```

```
      {metrics.map(([label, value]) => (
        <div key={label} className="glass rounded-3xl p-6">
          <div className="text-sm text-zinc-400">{label}</div>
          <div className="mt-2 text-3xl font-bold">{value}</div>
        </div>
      ))}
    </div>
  </DashboardShell>
 );
}
```

Deploy Steps

npm install
npm run dev

Then deploy:

vercel

What To Build Immediately After This

1. Replace cookie mock auth with real Supabase auth.
2. Persist Stripe customer IDs and plan status in Supabase.
3. Save agent configs and tool runs in the database.
4. Add worker payouts and review queues.
5. Add usage limits by plan.
6. Add affiliate links and referral tracking.
7. Add white-label custom branding for Enterprise.

---

53) Exact Next Prompt To Paste Into GitHub Copilot

Continue the Superagent Jroc AI repo.
Now implement real Supabase authentication, protected dashboard sessions, Stripe webhook subscription syncing, saved tool run history, saved agent records, verification worker applications, admin approval actions, and plan-based usage limits.
Keep the same premium dark SaaS design and preserve the existing file structure.

---

54) GitHub Codespaces Setup

Add these files so the repo opens and runs correctly in **GitHub Codespaces**.

.devcontainer/devcontainer.json

```
{
  "name": "Superagent Jroc AI Codespace",
  "image": "mcr.microsoft.com/devcontainers/javascript-node:1-20-bookworm",
  "customizations": {
    "vscode": {
      "settings": {
        "terminal.integrated.defaultProfile.linux": "bash",
        "editor.formatOnSave": true,
        "files.eol": "
"
      },
      "extensions": [
        "dbaeumer.vscode-eslint",
        "esbenp.prettier-vscode",
        "bradlc.vscode-tailwindcss",
        "ms-vscode.vscode-typescript-next",
        "GitHub.copilot",
        "GitHub.copilot-chat"
      ]
    }
  },
  "forwardPorts": [3000],
  "portsAttributes": {
    "3000": {
      "label": "Next.js App",
      "onAutoForward": "openPreview"
    }
  },
  "postCreateCommand": "npm install && cp .env.example .env.local || true",
  "remoteUser": "node"
}
```

.gitignore

```
node_modules
.next
.env.local
.env
```

```
.vercel
coverage
*.log
```

next.config.mjs

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    serverActions: {
      bodySizeLimit: '2mb'
    }
  }
};

export default nextConfig;
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["dom", "dom.iterable", "es2022"],
    "allowJs": false,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "baseUrl": ".",
    "paths": {
      "@/*": ["./*"]
    },
    "plugins": [{ "name": "next" ]
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules"]
}
```

Fix for tsconfig.json plugin line

Use this corrected version instead if needed:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["dom", "dom.iterable", "es2022"],
    "allowJs": false,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "baseUrl": ".",
    "paths": {
      "@/*": ["./*"]
    },
    "plugins": [{ "name": "next" }]
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules"]
}
```

postcss.config.js

```
module.exports = {
  plugins: {
    tailwindcss: {},
    autoprefixer: {}
  }
};
```

tailwind.config.ts

```
import type { Config } from 'tailwindcss';
```

```
const config: Config = {
  content: [
    './app/**/*.{js,ts,jsx,tsx,mdx}',
    './components/**/*.{js,ts,jsx,tsx,mdx}',
    './pages/**/*.{js,ts,jsx,tsx,mdx}'
  ],
  theme: {
    extend: {}
  },
  plugins: []
};

export default config;
```

next-env.d.ts

```
/// <reference types="next" />
/// <reference types="next/image-types/global" />

// NOTE: This file should not be edited.
```

.eslintrc.json

```
{
  "extends": ["next/core-web-vitals"]
}
```

app/api/health/route.ts

```
import { NextResponse } from 'next/server';

export async function GET() {
  return NextResponse.json({
    ok: true,
    service: 'Superagent Jroc AI',
    timestamp: new Date().toISOString()
  });
}
```

package.json

Use this safer Codespaces-ready version:

```
{
  "name": "superagent-jroc-ai",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev -H 0.0.0.0 -p 3000",
    "build": "next build",
    "start": "next start -H 0.0.0.0 -p 3000",
    "lint": "next lint"
  },
  "dependencies": {
    "@supabase/ssr": "latest",
    "@supabase/supabase-js": "latest",
    "lucide-react": "latest",
    "next": "14.2.30",
    "openai": "latest",
    "react": "18.3.1",
    "react-dom": "18.3.1",
    "stripe": "latest",
    "zod": "latest"
  },
  "devDependencies": {
    "@types/node": "latest",
    "@types/react": "latest",
    "@types/react-dom": "latest",
    "autoprefixer": "latest",
    "eslint": "latest",
    "eslint-config-next": "14.2.30",
    "postcss": "latest",
    "tailwindcss": "latest",
    "typescript": "latest"
  }
}
```

Terminal commands for Codespaces

```
npm install
cp .env.example .env.local
npm run dev
```

Then open the forwarded port for 3000.

---

55) Fast Copy-Paste Prompt for GitHub Copilot in Codespaces

Set up this Superagent Jroc AI repo to run perfectly in GitHub Codespaces.
Create .devcontainer/devcontainer.json, next.config.mjs, tsconfig.json, tailwind.config.ts, postcss.config.js, next-env.d.ts, .eslintrc.json, and update package.json so the app runs on port 3000 with host 0.0.0.0.
Make sure npm install works cleanly and the Next.js app launches inside Codespaces with forwarded port 3000.

---

56) Exact Folder To Create For Codespaces

```
superagent-jroc-ai/
  .devcontainer/
    devcontainer.json
  app/
  components/
  data/
  lib/
  db/
  public/
  types/
  .env.example
  .eslintrc.json
  .gitignore
  next-env.d.ts
  next.config.mjs
  package.json
  postcss.config.js
  tailwind.config.ts
  tsconfig.json
```